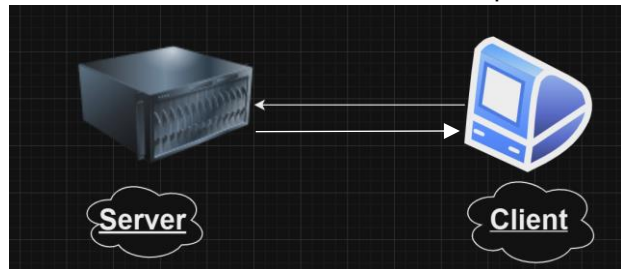


מטלה 3 – רשתות תקשורתThe Transport Layer

ראשית, נוסיף את הדיאגרמה שיצרנו באתר Draw.io על מנת להמחיש באופן ברור יותר את מה שמתרחש במטלה



במבט על, הפרויקט הזה בעצם דורש לרשום שתי תוכניות
 Server.py - צד השרת.
 Client.py - צד הלקוח.

חשוב לציין שעבדנו במערכת הפעלה של LINUX והקוד שם רץ ייתכן שבמערכת ההפעלה אחרת זה לא יעבוד.

מטרת המטלה היא לממש באופן בסיסי את הפרוטוקול תקשורת להעברת נתונים בהשראת פרוטוקול TCP. הפרוטוקול מתמקד במנגנונים מרכזיים של חלון הזזה, פילוח הודעות, וניהול שידורים חוזרים כדי להבטיח תקשורת אמינה בין הלקוח לשרת.

פילוח הודעות: חלוקה של הודעות גדולות למקטעים קטנים בהתאם לגודל מרבי שנקבע מראש.

ניהול חלון הזזה: שליחת מספר הודעות בו-זמנית, תוך שמירה על סדר נתונים וניהול הודעות מאושרות.

שידורים חוזרים על ידי טיימר: מנגנון שמבטיח שליחה מחדש של הודעות במקרה שהאישור להן לא התקבל בזמן.

במסגרת המטלה, נדרשנו לכתוב קוד בשפת Python שמבצע תקשורת מבוססת שקע (Socket) ומיישם את ההיגיון מאחורי יצירת חיבורים, שליחת נתונים ואישורם.

בדרישת המטלה צריך לקלוט מידע דרך שתי דרכים

1. דרך קלט מסוים שנכניס דרך המשתמש

2. דרך קובץ בפורמט הבא:

message:"This is a test message"

maximum_msg_size:400

window_size:4

timeout:5

בנוסף סוג הקובץ שאנחנו מעבירים הוא סיומת txt

אם הString ששלחתי הוא "This is a test message" וההודעה המקסימלית בגודל 5 אזי החלוקה תתבצע כך:

```

M0:Th
M1:is
M2: i
M3:s
M4:a
M5:te
M6:st
M7: m
M8:es
M9:sa
M10:g
M11:e
  
```

בפירוט הקבצים בנפרד נראה מה השימוש והמשמעות של המידע שקלטנו.

לפני שנתחיל להסביר על כל תוכנית בנפרד, נסביר על כמה מהמושגים שהשתמשנו בהם
 <--- maximum_msg_size > גודל של ההודעה המקסימלית שהשרת יכול לקבל, במידה שהקליינט ישלח הודעה גדולה
 ממה שנדרש נצטרך להשתמש בחלוקת ההודעה לחבילות קטנות יותר ולשלוח את ההודעה כך שהיא מפוצלת.
 <--- message > ההודעה שאותה נרצה לשלוח מהלקוח לשרת שלנו.
 <--- window_size > גודל חלון ההזזה כלומר חלון ההזזה קובע כמה הודעות ניתן לשלוח בלי לחכות לאישור מהשרת.
 <--- timeout > משמש להעברה מחודשת במידה שהחבילה או הACK אבדו או התעכבו.

Client.py

במבט כולל אנחנו דורשים מהקליינט לקרוא מהקובץ את הנתונים כמו timeout, message, maximum_msg_size ואותו דבר גם במידה שאנו קולטים מהמשתמש (*).
 בנוסף הגדרנו משתנים שונים על מנת שנוכל להגדיר את הלקוח שלנו

<--- SERVER_IP > משתנה המגדיר את כתובת הIP של השרת בדרך כלל זה יהיה 127.0.0.1
 <--- PORT > מספר הפורט שבו השרת מקבל חיבורים (בחירה הינה באופן רנדומלי בלבד).
 <--- BUFFER_SIZE > מציין כמה בתים ניתן לקרוא/לכתוב בכל פעם בתקשורת.
 <--- FILE_NAME > שם הקובץ ממנו ניקרא מידע במידה שנבחר לקלוט מידע מקובץ.

```
SERVER_IP = 'localhost'
PORT = 12348
BUFFER_SIZE = 1024
FILE_NAME = "input.txt"
```

לפני שנציג את הפונקציות שכתבנו נראה איך אנחנו בעצם מפעלים את הלקוח שלנו מפונקציה מרכזית הקוראת לכל
 שאר הפונקציות המפורטות בהמשך.

הפונקציה start_client אחראית להתחיל את פעולתו של הלקוח בתקשורת עם השרת. היא יוצרת חיבור מסוג TCP,
 שולחת ומקבלת הודעות, מבצעת פעולות על פי קלט מהמשתמש, ומסיימת את החיבור בסיום.

```
190 def start_client(): 1 usage
191 """
192     Initiates the client-side communication with the server.
193     1. Establishes a TCP connection to the server.
194     2. Sends the client's message to the server.
195     3. Receives the maximum allowed message size from the server.
196     4. Determines the input source (file or user input) and reads the message.
197     5. Splits the message into packets based on the maximum size.
198     6. Sends the packets using the sliding window technique, handling timeouts and acknowledgments.
199     7. Closes the connection after all messages are sent and acknowledged.
200 """
201 client_socket = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
202 try:
203     client_socket.connect((SERVER_IP, PORT))
204     print(f"Connected to the server at {SERVER_IP}:{PORT}")
205
206     # Decide File Or Input #
207     client_message = get_client_message()
208     client_socket.sendall(client_message.encode('utf-8'))
209     max_size = int(client_socket.recv(BUFFER_SIZE).decode('utf-8').strip())
210
211     print(f"Received maximum message size: {max_size} bytes")
212     if "File" in client_message:
213         message, window_size, timeout = read_input_from_file()
214     else:
215         message, window_size, timeout = read_input_from_user()
216     # End Decide File Or Input #
217     packets = split_message_to_packets_by_max_size(message, max_size)
218     send_in_sliding_window_method(client_socket, packets, window_size, timeout, max_size)
219     print(f"Finishing sending all messages + gets all acks back")
220     client_socket.close()
221 except Exception as e:
222     print(f"Error: {e}")
223
```

בשורה 201 אנחנו יוצרים סוקט עם חיבור של כתובת IPV4 עם חיבור אמין מסוג TCP ב
בשורה 203 הלקוח מתחבר לשרת באמצעות ה IP-והפורט שנמסרו לו. אם החיבור מצליח, הודעה מוצגת במסך
 שמציינת את ה IP-והפורט של השרת שאליו התחברנו.
בשורות 207-208 הפונקציה get_client_message אחראית לקבל קלט מהמשתמש או לקרוא את המידע שצריך
 לשלוח לשרת.
 לאחר מכן, ההודעה נשלחת לשרת באמצעות sendall, שהיא פונקציה ששולחת את כל ההודעה בבת אחת.

בשורה 209 הלקוח מקבל מהשרת את הגודל המרבי המותר להודעות שמותר לשלוח. ולאחר מכן קולטים את שאר הנתונים שלנו ומשמשים בפונקציות עזר על מנת לממש את הדרוש.

הפונקציות המופיעות בקובץ :

- יצרנו את הפונקציה הבאה על מנת להבין מאיפה אנחנו רוצים לקלוט קלט , מהמשתמש או מקריאה מקובץ , כלומר יצרנו תפריט המייצג את האפשרויות 1 או 2
- 1 --- אם נרצה לקבל מידע מהמשתמש. (*)
- 2 --- אם נרצה לקבל את המידע מקריאה מקובץ. (*)

```
def get_client_message():
    """
    Displays a menu to the client to choose an input method and returns the corresponding request message.
    This function prompts the client with two options:
    1. To provide data manually via input.
    2. To provide data via a file.
    - If the client enters "1", returns a message to request the data via input.
    - If the client enters "2", returns a message to request the data from file.
    """
    print("Choose one option:")
    print("1. Please provide data from Input")
    print("2. Please provide data from File")
    choice = input("Enter your choice (1 or 2): ").strip()
    if choice == "1":
        return "Please provide data from Input"
    elif choice == "2":
        return "Please provide data from File"
```

- יצרנו את הפונקציה הבאה על מנת לאפשר ללקוח לקלוט מידע מהמשתמש (*).

```
def read_input_from_user():
    """
    Prompts the user to enter message details and returns the input values.
    The function collects the following information from the user:
    - A message string.
    - The window size (number of messages) as an integer.
    - The timeout value (in seconds) as an integer.
    """
    message = input("Enter the message: ")
    window_size = int(input("Enter the window size (number of messages): "))
    timeout = int(input("Enter the timeout (in seconds): "))
    return message, window_size, timeout
```

- יצרנו את הפונקציה הבאה על מנת לאפשר ללקוח לקרוא מידע מקובץ (*).

```
def read_input_from_file():
    with open(FILE_NAME, 'r') as file:
        lines = file.readlines()
        message = ""
        window_size = 0
        timeout = 0
        for line in lines:
            if "message:" in line:
                message = line.split(":")[1].strip().strip('\"')
            elif "window_size:" in line:
                window_size = int(line.split(":")[1].strip())
            elif "timeout:" in line:
                timeout = int(line.split(":")[1].strip())
        return message, window_size, timeout
```

הפונקציות הבאות עוסקות במימוש של ה Sliding Window:

הפונקציה הבאה מפצלת הודעה למנות בגודל מקסימלי מוגדר, כאשר לכל חבילה יש קידומת אינדקס על מנת שנוכל

לשמור על הסדר בשליחת ההודעה.

```

75 def split_message_to_packets_by_max_size(message, max_size): 1usage
76 """
77 Splits a message into packets, where each packet does not exceed a specified maximum size.
78 Arguments:
79     message (str): The message to be split into packets.
80     max_size (int): The maximum size allowed for each packet.
81 Returns:
82     list: A list of packets, each containing a portion of the original message, prefixed with its index.
83 """
84 encoded_message = message.encode()
85 results_packets = []
86 start_index = 0
87 packet_index = 0
88
89 while start_index < len(encoded_message):
90     prefix_packet = f"H{packet_index}:"
91     available_size = max_size - len(prefix_packet.encode("utf-8"))
92     if available_size <= 0:
93         print("No available size")
94         break
95     chunk = encoded_message[start_index: start_index + available_size]
96     results_packets.append(f"{prefix_packet}{chunk.decode()}")
97     start_index += len(chunk)
98     packet_index += 1
99
100 return results_packets

```

הפונקציה הבאה מיישמת את פרוטוקול החלון הזהה לשליחת מנות וקבלת אישורים. **שורה 72** הקטע עם המערך של ה-False משמש כדי לעקוב אחרי מצב האישור (ACK) של כל חבילת נתונים שנשלחת. המערך הזה נקרא received_ack_packets והוא מכיל רשימה של ערכים בוליאניים (True/False) כאשר כל ערך מייצג חבילה בודדת ומצב האישור שלה.

```

59 send_in_sliding_window_method(client_socket, packets, window_size, time_out, max_size): 1usage
60 """
61 Implements the sliding window protocol to send packets and wait for their acknowledgments.
62 Arguments:
63     client_socket (socket): The socket to send and receive data over.
64     packets (list): A list of packets to be sent.
65     window_size (int): The number of packets to send in one window.
66     time_out (float): The timeout period in seconds.
67     max_size (int): The maximum size for an ACK message.
68 Returns:
69     None: This function handles the transmission and acknowledgment of packets.
70 """
71 packets_size = len(packets)
72 received_ack_packets = [False] * packets_size
73 current_start_windows_index = 0
74
75 while current_start_windows_index < packets_size:
76     send_packets_in_window(client_socket, packets, received_ack_packets, current_start_windows_index, window_size)
77     start_receiving_acks_with_timeout(client_socket, time_out, received_ack_packets, max_size, window_size)
78     last_window_size = current_start_windows_index
79     while current_start_windows_index < packets_size and received_ack_packets[current_start_windows_index]:
80         current_start_windows_index += 1
81     if last_window_size != current_start_windows_index:
82         print(f"window is move to index {current_start_windows_index}")
83

```

הפונקציה הבאה מקשיבה לאישורים מהשרת, עם מנגנון זמן קצוב במקום. הפונקציה עוקבת אחר הזמן שחלף (start_time ו-resting_time_out) ומאזינה להודעות ACK נכנסות בתוך הזמן הקצוב הנתון. הודעת ACK של השרת מנותחת כדי לחלץ את מספר האישור. רשימת receive_ack_packets מתעדכנת כדי לסמן את החבילות המתאימות כמושורות. אם לא התקבל ACK בתוך הזמן הקצוב, או שמתרחשת שגיאה, הפונקציה מדפיסה הודעה ויוצאת.

```

28 def start_receiving_acks_with_timeout(client_socket, time_out, received_ack_packets, max_size, window_size):
39     """
40     start_time = time.time()
41     remaining_time_out = time_out
42     number_of_received_packets = 0
43     while remaining_time_out > 0 and number_of_received_packets < window_size:
44         try:
45             elapsed_time = time.time() - start_time
46             remaining_time_out = time_out - elapsed_time
47             if remaining_time_out <= 0:
48                 print("General timeout reached.")
49                 break
50
51             client_socket.settimeout(remaining_time_out)
52             ack_from_server = client_socket.recv(max_size).decode()
53             stripped_data = ack_from_server.rstrip('#')
54             ack_number = int(stripped_data[3:])
55             for j in range(ack_number + 1):
56                 if not received_ack_packets[j]:
57                     number_of_received_packets += 1
58                     print(f"Received ACK{ack_number}")
59                     received_ack_packets[j] = True
60             except socket.timeout:
61                 print("Timeout reached while waiting for messages.")
62                 break
63             except Exception as e:
64                 print(f"An error occurred: {e}")
65                 break
66         if remaining_time_out <= 0:
67             print(f"timeout is occurred")
68

```

בשורה 53 אנחנו משתמשים בהורדה של # מכל הACK שנקבל, את זה נסביר בחלק הSERVER מדוע עשינו את זה ורק אחרי שקיבלנו ACK אפשר להשים במערך TRUE ולאחר מכן להמשיך הלאה.

הפונקציה הבאה אחראית על שליחת מנות בתוך חלון ספציפי לשרת, תוך בדיקה אם מנות כלשהן כבר אושרו. הלולאה פועלת מה-current_start_windows_index עד למינימום של current_start_windows_index + window_size והמספר הכולל של מנות. אם חבילה באינדקס i לא אושרה (received_ack_packets[i] היא False), החבילה נשלחת דרך הסוקט ומודפסת.

```

21 def send_packets_in_window(client_socket, packets, received_ack_packets, current_start_windows_index, window_size):
22     """
23     Sends packets in a specific window and checks whether any of the packets have already been acknowledged.
24     Arguments:
25         client_socket (socket): The socket to send the packets over.
26         packets (list): A list of packets to send.
27         received_ack_packets (list): A list tracking whether each packet has been acknowledged.
28         current_start_windows_index (int): The index from where to start sending packets.
29         window_size (int): The number of packets to send in one window.
30     """
31     for i in range(current_start_windows_index, min(current_start_windows_index + int(window_size), len(packets))):
32         if not received_ack_packets[i]:
33             print(f"Sending packet {i}: {packets[i]}")
34             client_socket.send(packets[i].encode())
35

```

הפונקציה הבאה מפצלת רשימה של מנות לחלונות בגודל מוגדר. כלומר זה מאתחל חלונות רשימה ריקה כדי לאחסן את חלונות החבילות. total_packets מכיל את המספר הכולל של מנות ברשימת הקלט.

```

10 def split_into_windows(packets, window_size):
11     """
12     Splits a list of packets into windows of a specified size.
13     Arguments:
14         packets (list): A list of packets to be split.
15         window_size (int): The number of packets in each window.
16     Returns:
17         list: A list of windows, where each window is a sublist containing a subset of the packets.
18     """
19     windows = []
20     total_packets = len(packets)
21
22     for i in range(0, total_packets, window_size):
23         window = packets[i:i + window_size]
24         windows.append(window)
25
26     return windows

```

כמו שאפשר לראות לכל פונקציה יש תפקיד בשליחת וקבלת נתונים בפרוטוקול חלון הזזה. חלון ההזזה מבטיח שהמנות נשלחות בחבילות, כאשר כל חבילה מקבלת אישור לפני שתמשיך לשלוח חבילות.

כעת נדבר על ה Server.py:

המידע על maximum_msg_size עלינו לקבל מהשרת ולכן נחלק למקרים מקרה ראשון: קריאה מהקובץ, אזי עבורנו השרת יקרא מהקובץ וישלח את גודל ההודעה המקסימלי ללקוח על מנת שהלקוח ידע איזה גודל של הודעה לשלוח (*).
מקרה שני: אנחנו נכניס דרך הקונסול את גודל המידע המקסימלי שהשרת יכול לקבל ואז השרת יעביר את מה שהכנסנו ללקוח (*).
ניכנס פנימה להגדרות שנתנו לצד השרת:

```
1 import socket
2
3 PORT = 12348
4 SERVER_ADDRESS = ('', PORT)
5 MAX_CLIENTS = 1
6 FILE_NAME = "input.txt"
```

SERVER_ADDRESS < --- כתובת ה IP-והפורט של השרת. כלומר כאן מצוין שהשרת יהיה זמין לכל כתובת IP במחשב המקומי (החיבור יכול להתקבל מכל כתובת). החלק הראשון הוא כתובת ה IP-של השרת והחלק השני הוא הפורט שבו השרת יאזין לחיבורים.

PORT < --- מספר הפורט שהשרת ישתמש בו על מנת להאזין לחיבורים נכנסים. הפורט הזה מציין את הכתובת הפיזית בה השרת יאזין לחיבורים מלקוח.

FILE_NAME < --- שם הקובץ ממנו ניקרא מידע במידה שנבחר לקלוט מידע מקובץ.

MAX_CLIENTS < --- מספר המקסימלי של לקוחות שמותר לשרת לקבל בו זמנית. במקרה הזה, השרת מוכן לקבל חיבור אחד בלבד בו זמנית. זה אומר ששרת זה יטפל בלקוח אחד בלבד בכל זמן נתון. אם יתחברו לקוחות נוספים, הם ימתינו עד שהלקוח הנוכחי יסיים את החיבור.

לפני שנתחיל לפרט על הפונקציות שלנו, נראה את הפונקציה ההתחלתית והמרכזית שלנו:
הפונקציה start_server אחראית להפעיל את השרת, להמתין לחיבור לקוחות, ולטפל בבקשותיהם

```
115 def start_server(): 1 usage
116
117 """
118 Starts the server, listens for incoming client connections, and processes client requests.
119 The function creates a server socket, binds it to the specified address, and listens for incoming
120 connections. When a client connects, it handles the client's request by calling the 'handle_client' function.
121 The server continuously waits for new connections and processes them until an error occurs or the server is shut down.
122 Steps:
123 1. Create and bind the server socket to the specified address.
124 2. Wait for a client connection.
125 3. Accept the client connection and handle it.
126 4. Close the client socket and wait for the next client.
127 """
128 server_socket = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
129 try:
130     server_socket.bind(SERVER_ADDRESS)
131     print(f"Server started and listening on ${SERVER_ADDRESS}")
132     server_socket.listen(MAX_CLIENTS)
133
134     while True:
135         print("Waiting for a client to connect...")
136         client_socket, client_address = server_socket.accept()
137         print(f"Connection established with {client_address}")
138         handle_client(client_socket)
139         client_socket.close()
140 except Exception as e:
141     print(f"Error: {e}")
142 finally:
143     server_socket.close()
144     print("Server shut down.")
```

בשורה 127 הפונקציה יוצרת אובייקט socket חדש שנועד לתקשורת באמצעות פרוטוקול TCP/IP (SOCK_STREAM) עם כתובת IP ופורט שהוגדרו מראש בSERVER_ADDRESS

בשורה 129-131 השרת מחבר את ה socket-לכתובת שהוקצתה לו כלומר לפי כתובת ה IP-והפורט, ואז מתחיל להאזין לחיבורים נכנסים עם הגדרת מספר הלקוחות המקסימלי שיכול להתחבר במקביל.

בשורה 135 ברגע שהשרת מוכן, הוא ממתין לחיבור מלקוח באמצעות accept() כשהחיבור מתבצע, נוצר אובייקט

client_socket שמייצג את החיבור עם הלקוח.
לאחר קבלת החיבור, הפונקציה handle_client נקראת כדי לטפל בקשר עם הלקוח ולאחר סיום התקשורת נסגור את הסוקט שיצרנו

הפונקציה הבאה מניבה את היכולת לקבל קלט מקובץ כלומר את היכולת לקרוא מקובץ בשרת שלנו (*).

```
19 def handle_max_size_from_file(): 1 usage
20     """
21     Reads the maximum allowed message size from a file.
22     The function opens the specified file (defined by the global 'FILE_NAME'),
23     reads its lines, and searches for the line containing the "maximum_msg_size:" keyword.
24     It then extracts and returns the value following the colon as the maximum message size.
25     """
26     with open(FILE_NAME, 'r') as file:
27         lines = file.readlines()
28         for line in lines:
29             if "maximum_msg_size:" in line:
30                 return line.split(":")[1].strip()
```

הפונקציה הבאה מניבה את היכולת לקבל קלט מהמשתמש בשרת שלנו (*).

```
9 def handle_max_size_from_input(): 1 usage
10     """
11     Prompts the user to input the maximum allowed message size in bytes.
12     The function waits for the user to input a value, converts the input to an integer,
13     and returns it as the maximum message size.
14     """
15     max_size = int(input("Enter the maximum message size (in bytes): "))
16     return max_size
```

דרך הפונקציה הבאה אנחנו בודקים מה הלקוח שלח ודרך זה מחזירים את התשובה של הגודל של ההודעה המקסימלית

```
32 def get_and_handle_first_max_size_message(client_message): 1 usage
33     """
34     Handles the request for the maximum message size based on the client's message.
35     The function checks the provided 'client_message' and calls either
36     'handle_max_size_from_input' or 'handle_max_size_from_file' accordingly.
37     - If the message is "Please provide data from Input", it prompts the user for input.
38     - If the message is "Please provide data from File", it reads the size from a file.
39     """
40     if client_message == "Please provide data from Input":
41         return handle_max_size_from_input()
42     elif client_message == "Please provide data from File":
43         return handle_max_size_from_file()
```

הפונקציה extract לוקחת רצף של התווים שמצורף לו מזהה חבילה ונתון תוכן, ומבצעת את הפעולות הבאות:
הפונקציה מפרקת את רצף התווים לשני חלקים בעזרת הפונקציה split(":"), כמאשר החלק הראשון יהיה מזהה החבילה (packet identifier) והחלק השני יהיה תוכן ההודעה. (message content)
מתוך המזהה, הפונקציה מחלצת את מספר החבילה על ידי חיתוך הטקסט (packet_identifier[1:]) והמרתו למספר שלם. (int) נניח אם המזהה היה "M1", הפונקציה תחזיר את המספר 1.

```
67 def extract(decoded_msg): 1 usage
68     """
69     Extracts the packet number and message content from a decoded message.
70     The function splits the input message into two parts:
71     1. The packet identifier which is used to determine the packet number.
72     2. The message content following the (":").
73     """
74     packet_identifier, message_content = decoded_msg.split(":", 1)
75     packet_number = int(packet_identifier[1:])
76     return packet_number, message_content
```


הפונקציה הבאה :

handle_client אחראית לטפל בתקשורת בין השרת ללקוח לאחר קבלת חיבור מלקוח.
נתחיל בחלקה הראשון של הפונקציה

```

57 def handle_client(client_socket): 1 usage
58     """
59     Handles the communication with a connected client.
60     The function receives a message from the client to determine the maximum allowed
61     message size. It then continuously receives packets from the client, processes
62     them in order, and sends back acknowledgment (ACK) messages. If packets arrive
63     out of order, the server waits for the expected packet.
64     Steps:
65     1. Receive and process client message to get the maximum message size.
66     2. Continuously receive packets from the client.
67     3. Extract packet number and message content.
68     4. Store and reassemble packets in order.
69     5. Send acknowledgment (ACK) messages back to the client
70     """
71     try:
72         client_message = client_socket.recv(1024).decode('utf-8').strip()
73         max_size = get_and_handle_first_max_size_message(client_message)
74         max_size_number = int(max_size)
75         client_socket.sendall(f"{max_size}".encode('utf-8'))
76         print(f"Sending back maximum message size: {max_size}")
77         packets_received = []
78         packet_expected_from_client = 0
79         while True:
80             msg_from_client = client_socket.recv(max_size_number)
81             decoded_msg = msg_from_client.decode()
82             if not decoded_msg.strip():
83                 print("Client is finished , Closing socket")
84                 break
85             print(f"Server received message - {decoded_msg}")
86             packet_number, message_content = extract(decoded_msg)

```

נביט בשורה 72 הפונקציה מקבלת נתונים מהלקוח דרך ה (client_socket.recv(1024)), עד 1024 בתים.
המידע שנקרא ממומר לטקסט ('utf-8').decode() ומנוקה ממחרוזות ריקות בקצוות בעזרת strip().
בשורות 73-75 הפונקציה בודקת את הודעת הלקוח. אם הלקוח ביקש גודל מרבי להודעות, אז
get_and_handle_first_max_size_message מחליטה אם לקרוא את הערך מקובץ או לבקש מהמשתמש להזין אותו.
לאחר מכן, הגודל המרבי מומר למספר שלם ומחזיר ללקוח כדי לוודא שהלקוח יודע את המגבלה.
שורה 86 ההודעה מהלקוח מפורקת באמצעות הפונקציה extract כדי לחלץ את מספר החבילה ואת תוכן ההודעה.
החלק השני של הפונקציה :

```

87
88     if packet_number >= len(packets_received):
89         packets_received.extend([None] * (packet_number - len(packets_received) + 1))
90
91     if packets_received[packet_number] is None:
92         packets_received[packet_number] = message_content
93
94     if packet_number == packet_expected_from_client:
95         print(f"Received expected packet {packet_number}")
96         packet_expected_from_client += 1
97         while packet_expected_from_client < len(packets_received) and packets_received[
98             packet_expected_from_client] is not None:
99             packet_expected_from_client += 1
100     else:
101         print(
102             f"Out-of-order packet {packet_number} received, still waiting for packet {packet_expected_from_client}")
103
104     ack_message = f"ACK{packet_expected_from_client - 1}"
105     ack_message = ack_message.ljust(max_size_number, _fillchar: '#')
106     print(f"Sending ack message {ack_message}")
107     client_socket.send(ack_message.encode())
108 except Exception as e:
109     print(f"Error handling client: {e}")

```

שורות 88 89 השרת שומר את התוכן שהתקבל במערך packets_received כך שכל חבילה תקבל את המיקום שלה

לפי מספר החבילה שלה. אם מספר החבילה שונה ממספר החבילה הבא (במידה והתקבלו חבילות לא בסדר), השרת מאריך את המעריך כדי להוסיף מקום לכל החבילות.
שורות 104-107 השרת שולח הודעת אישור (ACK) ללקוח עם מספר החבילה שהתקבלה לאחרונה. הודעת ה-ACK ממולאת בסימני # כדי להתאים לגודל המרבי של ההודעה מה שמוודא שההודעה של ה-ACK תהיה בגודל הרצוי.

ובאופן כללי נתאר כיצד מימשנו את המנגנונים בהם עוסק התרגיל, מבנה ההודעה כולל שדות ההודעה וגדלים:

- ההודעה שנשלחת בין הלקוח לשרת מחולקת לחבילות (packets) כך שכל חבילה מכילה חלק מההודעה המקורית.
- כל חבילה מקודדת עם מזהה שמציין את המספר שלה למשל ("M0:", "M1:")
- ישנה חלוקה של ההודעה לפי גודל מקסימלי שמוגדר מראש (בהתאם לדרישות הלקוח או הקובץ).
- הלקוח שולח את ההודעה בקטעים שנקראים **חבילות**, כשהן מעוברות בסדר רציף עד שהן מגיעות לשרת. כל חבילה מקבלת אישור קבלה (ACK) מהשרת.
- הפרוטוקול של חלון ההזזה מאפשר שליחה של מספר חבילות בו זמנית, כאשר הלקוח שולח קבוצת חבילות (window) והשרת מאשר את קבלתן.
- הלקוח ממתין לאישור (ACK) לפני שהוא שולח את החבילות הבאות.
- כל חבילה שנשלחת מקבלת אישור מהשרת בצורת הודעת ACK עם מספר החבילה למשל, "ACK0", "ACK1", אם החבילה הגיעה בזמן והמספר שלה מתאים, השרת שולח ACK עם המספר המתאים.
- אם החבילה לא התקבלה, הלקוח ימשיך לשלוח את החבילות שטרם קיבלו אישור.
- כל הודעה (packet) מכילה את המידע הבא:

מזהה החבילה: לדוגמה "M0", "M1", שמייצג את מספר החבילה.

תוכן ההודעה: החלק מההודעה המקורית שמועבר בחבילה.

(הארה: ולכן כשנכניס MAXSIZE נצטרך להתחיל לפחות מ2 על מנת שיהיה מקום לזיהוי החבילה וגם לתוכן.)

- כל הודעת ACK מכילה את המידע הבא:
- ACK** : המזהה שמציין את החבילה שהתקבלה למשל "ACK0"

- הגודל של כל חבילה מוגדר לפי הערך, max_size, שהוא הערך המקסימלי של גודל ההודעה שהשרת או הלקוח יכולים להתמודד איתו.

- כל הודעה מחולקת לחבילות בהתאם לגודל זה, כך שהחבילות לא יחרגו מהגודל המותר.
- הלקוח שולח חבילות בפרקי זמן קבועים וממתין לאישור מהשרת. אם לא מתקבל אישור בזמן שנקבע, המנגנון מציין שהייתה חריגה בזמן, (Timeout) והלקוח חוזר לשלוח את החבילות שעדיין לא אושרו.
- אם השרת מקבל חבילה מחוץ לסדר (למשל, חבילה עם מספר גבוה יותר ממה שמצופה), הוא שומר אותה וממתין לחבילה הקודמת כדי להשלים את הסדר.

כעת נסתכל על המימוש עצמו :

המימוש עצמו :

1. התחברות לשרת:

הלקוח מתחבר לשרת דרך סוקט TCP ומבצע שליחה של בקשה לבחירת מקור נתונים (האם להכניס ידנית או לקרוא מקובץ).

2. הגדרת המגבלה על גודל ההודעה:

השרת שולח בחזרה את המגבלה על גודל ההודעה המקסימלי (max_size) שניתן לשלוח בבת אחת. הלקוח מקבל את הערך הזה ומחשב את החבילות על פי המגבלה.

3. פיצול ההודעה לחבילות: (Packets)

הלקוח מפצל את ההודעה לחבילות קטנות. כל חבילה מקבלת מזהה

המערך received_ack_packets משמש כדי לעקוב אחרי מצב החבילות שנשלחו לשרת ולוודא אם התקבלו עבורן אישורים (ACK) מהשרת.

- גודל המערך: שווה למספר החבילות. (len(packets)).

- ערכי המערך:

False החבילה לא קיבלה עדיין אישור.

True החבילה קיבלה אישור מהשרת.

4. שליחת חבילות לפי: Sliding Window

שלב 1: הלקוח שולח חבילות בשורות (חלונות) בגודל שנבחר. (window_size) הוא שולח חבילות אחת אחרי השנייה בתוך החלון, תוך שהוא בודק אם כל חבילה קיבלה אישור. (ACK)

שלב 2: אם חבילה לא קיבלה אישור לפי ACK או אם חלף הזמן, (timeout) הלקוח שולח שוב את החבילות שלא אושרו.

שלב 3: הלקוח ממתין לאישור (ACK) מהשרת. כל הודעת ACK מחזירה את מספר החבילה שהתקבלה ומעבירה את המצב להודעה הבאה. במידה והחבילה לא הגיעה בסדר הנכון, השרת מחכה לחבילה החסרה.

5. המתנה לאישורים: (ACKs)

הלקוח משתמש בטיימר (timeout) כדי להמתין לאישורים. אם זמן ההמתנה עבר ולא התקבל אישור, הלקוח ישלח מחדש את החבילות שלא אושרו.

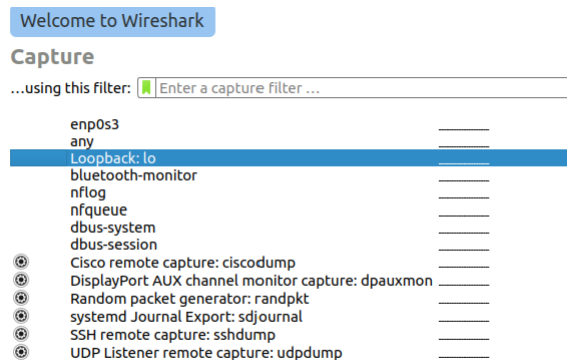
6. שליחת הודעות ACK מהשרת:

השרת בודק את סדר החבילות שהתקבלו. אם חבילה התקבלה מחוץ לסדר, היא נשמרת עד שתתקבל החבילה הקודמת. השרת שולח הודעת ACK עם מספר החבילה שהתקבלה. כל ACK מכילה את המספר של החבילה שהתקבלה בפועל) למשל. "ACK0"

7. שלב סיום:

כל החבילות התקבלו והלקוח מקבל אישור ששלח את כל ההודעה בהצלחה. הלקוח סוגר את החיבור.

כעת נתעסק בהקלטת תעבורת המידע, בWireshark :

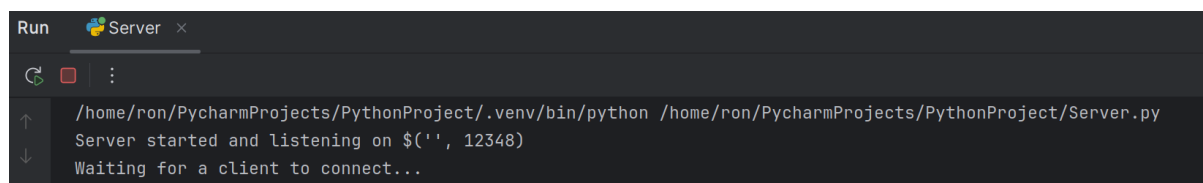


נבחר ב `Loopback:lo` משום שההודעות שלנו עוברות דרך המחשב לא דרך אמצעי חיצוני.

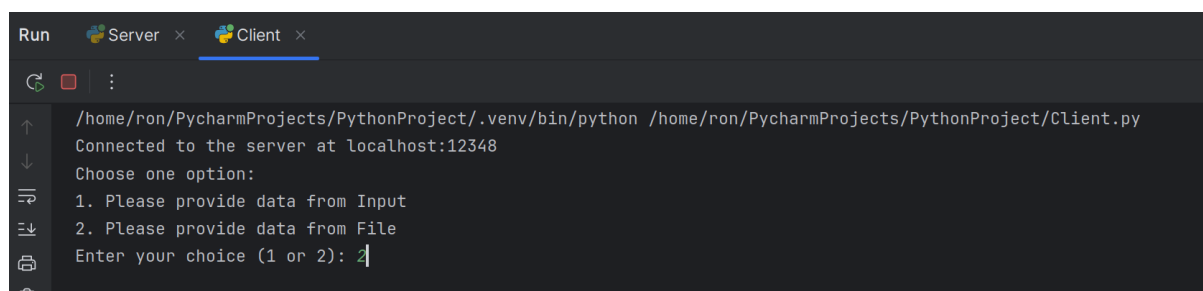
מטרת ההקלטה הבאה היא להראות את התעבורה כשהלקוח שולח הודעה שקטנה מה `max_size`, נפעיל את ההקלטה נתחיל במקרה שבו ההודעה שנרצה לשלוח קטנה מ `max_size` על מנת לבדוק שהכל עובד כראוי נקלוט מידע מהקובץ הבא:

```
message:"This is a test message"
maximum_msg_size:400
window_size:4
timeout:5
```

נפעיל את השרת:



נפעיל את הלקוח ונבחר באופציה 2:



בצד הלקוח:

```

/home/ron/PycharmProjects/PythonProject/.venv/bin/python /home/ron/PycharmProjects/PythonProject/Server.py
Connected to the server at localhost:12348
Choose one option:
1. Please provide data from Input
2. Please provide data from File
Enter your choice (1 or 2): 2
Received maximum message size: 400 bytes
Sending packet 0: M0:" This is a test message"
Received ACK0
Timeout reached while waiting for messages.
window is move to index 1
Finishing sending all messages + gets all acks back

```

בצד השרת:

```

Run Server Client
/home/ron/PycharmProjects/PythonProject/.venv/bin/python /home/ron/PycharmProjects/PythonProject/Server.py
Server started and listening on $('', 12348)
Waiting for a client to connect...
Connection established with ('127.0.0.1', 41620)
Sending back maximum message size: 400
Server recived message - M0:" This is a test message"
Received expected packet 0
Sending ack message ACK0#####
Client is finished , Closing socket
Waiting for a client to connect...

```

נשתמש במנגנון הסינון ונרשום בו tcp :

The image shows a Wireshark packet capture of a TCP connection. The top section displays a list of packets with their details expanded. The bottom section shows the same packets with Hebrew annotations explaining the handshake process:

- סיים הקשר עם הלקוח** (Finished connection with client): Points to the final ACK packet (Seq=56, Ack=405).
- הודעה שרצינו להעביר** (Message we wanted to send): Points to the PSH packet (Seq=30, Ack=4).
- השרת החזיר את max size** (Server returned max size): Points to the ACK packet (Seq=4, Ack=55).
- בקשה של הנתונים** (Request for data): Points to the PSH packet (Seq=55, Ack=404).
- Handshake packets**: Points to the SYN and ACK packets (Seq=0, Ack=1 and Seq=1, Ack=30).

אפשר לראות דרך ההקלטה את לחיצת היד המשולשת של ה TCP (החלק האדום) ובנוסף את התקשורת בניהם, ראשית הקליינט שולח לשרת מה הגודל המקסימלי של ההודעה שהוא יכול לקבל, את המידע הזה השרת מקבל מהקובץ (בדוגמה זו ספציפית).

(החלק התכלת) אפשר לראות את מה שהלקוח שלח לשרת

0000	00 00 00 00 00 00 00 00 00 00 00 00 08 00 45 00	E.
0010	00 51 0e e6 40 00 40 06 2d bf 7f 00 00 01 7f 00	Q..@. -.....
0020	00 01 d3 56 30 3c 4b 65 9a 93 88 17 f5 b1 80 18	..V0<Ke
0030	02 00 fe 45 00 00 01 01 08 0a ae 7c 53 b0 ae 7b	..E..... S..{
0040	67 7f 50 6c 65 61 73 65 20 70 72 6f 76 69 64 65	g>Please provide
0050	20 64 61 74 61 20 66 72 6f 6d 20 46 69 6c 65	data fr om File

השרת מחזיר 400 (החלק הצהוב)

Data (3 bytes)		
Data: 343030		
[Length: 3]		
0000	00 00 00 00 00 00 00 00 00 00 00 00 08 00 45 00	E.
0010	00 37 16 2a 40 00 40 06 26 95 7f 00 00 01 7f 00	7.*@. &.....
0020	00 01 30 3c d3 56 88 17 f5 b1 4b 65 9a b0 80 18	..0<V...Ke...
0030	02 00 fe 2b 00 00 01 01 08 0a ae 7c 53 b0 ae 7c	...+..... S..
0040	53 b0 34 30 30	S.400

כאן נוכל לראות את ההודעה שהלקוח שלח לשרת (החלק הוורוד)

0000	00 00 00 00 00 00 00 00 00 00 00 00 08 00 45 00	E.
0010	00 4d 0e e8 40 00 40 06 2d c1 7f 00 00 01 7f 00	M..@. -.....
0020	00 01 d3 56 30 3c 4b 65 9a b0 88 17 f5 b4 80 18	..V0<Ke
0030	02 00 fe 41 00 00 01 01 08 0a ae 7c 53 b1 ae 7c	..A..... S..
0040	53 b0 4d 30 3a 54 68 69 73 20 69 73 20 61 20 74	S:M0:Thi s is a t
0050	65 73 74 20 6d 65 73 73 61 67 65	est mess age

בחלק הסגול הקשר פשוט נסגר . ההקלטה למקרה הזה מופיעה בשם q1.

ההקלטה הבאה מדמה את המצב בו אנחנו שולחים הודעה שהיא גדולה מהגודל המקסימלי :

כעת נבחן את המקרה בו אנחנו רוצים לשלוח מלקוח לשרת הודעה שהיא גדולה מהגודל המקסימלי שהשרת יכול לקבל ולכן נצטרך לחלק את ההודעה ואלו הנתונים ששלחנו

1	message:"This is a test message"
2	maximum_msg_size:5
3	window_size:2
4	timeout:2

אז , נפעיל את השרת והלקוח וננתח את המידע המעובר בWireshark :

צד הלקוח :

```

Connected to the server at localhost:12348
Choose one option:
1. Please provide data from Input
2. Please provide data from File
Enter your choice (1 or 2): 2
Received maximum message size: 5 bytes
Sending packet 0: M0:Th
Sending packet 1: M1:is
Received ACK0
Received ACK1
window is move to index 2
Sending packet 2: M2: i
Sending packet 3: M3:s
Received ACK2
Received ACK3
window is move to index 4
Sending packet 4: M4:a
Sending packet 5: M5:te
Received ACK4
Received ACK5
window is move to index 6
Sending packet 6: M6:st
Sending packet 7: M7: m
Received ACK6
Received ACK7
window is move to index 8
Sending packet 8: M8:es
Sending packet 9: M9:sa
Received ACK8
Received ACK9
window is move to index 10

```

```

Sending packet 10: M10:g
Sending packet 11: M11:e
Received ACK10
Received ACK11
window is move to index 12
Finishing sending all messages + gets all acks back

```

צד השרת:


```

server started and listening on 55774
Waiting for a client to connect...
Connection established with ('127.0.0.1', 55774)
Sending back maximum message size: 5
Server recived message - M0:Th
Received expected packet 0
Sending ack message ACK0#
Server recived message - M1:is
Received expected packet 1
Sending ack message ACK1#
Server recived message - M2: i
Received expected packet 2
Sending ack message ACK2#
Server recived message - M3:s
Received expected packet 3
Sending ack message ACK3#
Server recived message - M4:a
Received expected packet 4
Sending ack message ACK4#
Server recived message - M5:te
Received expected packet 5
Sending ack message ACK5#
Server recived message - M6:st
Received expected packet 6
Sending ack message ACK6#
Server recived message - M7: m
Received expected packet 7
Sending ack message ACK7#
Server recived message - M8:es
Received expected packet 8
Sending ack message ACK8#
Received expected packet 9
Sending ack message ACK9#
Server recived message - M10:g
Received expected packet 10
Sending ack message ACK10
Server recived message - M11:e
Received expected packet 11
Sending ack message ACK11
Client is finished , Closing socket
Waiting for a client to connect...

```

נביט באWireshark:

No.	Time	Source	Destination	Protocol	Length	Info
1	0.000000	127.0.0.1	127.0.0.1	TCP	74	55774 → 12348 [SYN] Seq=0 Win=65536 Len=0 MSS=65536 SACK_PERM=1 TSval=2930220951 TSecr=0 WS=128
2	0.000073	127.0.0.1	127.0.0.1	TCP	74	12348 → 55774 [SYN, ACK] Seq=0 Ack=1 Win=65536 Len=0 MSS=65536 SACK_PERM=1 TSval=2930220951 TSecr=2930220951 WS=128
3	0.000091	127.0.0.1	127.0.0.1	TCP	66	55774 → 12348 [ACK] Seq=1 Ack=1 Win=65536 Len=0 TSval=2930220951 TSecr=2930220951
4	2.225953	127.0.0.1	127.0.0.1	TCP	95	55774 → 12348 [PSH, ACK] Seq=1 Ack=1 Win=65536 Len=29 TSval=2930223177 TSecr=2930220951
5	2.225968	127.0.0.1	127.0.0.1	TCP	66	12348 → 55774 [ACK] Seq=1 Ack=30 Win=65536 Len=0 TSval=2930223177 TSecr=2930223177
6	2.226236	127.0.0.1	127.0.0.1	TCP	67	12348 → 55774 [PSH, ACK] Seq=1 Ack=30 Win=65536 Len=1 TSval=2930223178 TSecr=2930223177
7	2.226241	127.0.0.1	127.0.0.1	TCP	66	55774 → 12348 [ACK] Seq=30 Ack=2 Win=65536 Len=0 TSval=2930223178 TSecr=2930223178
8	2.226406	127.0.0.1	127.0.0.1	TCP	71	55774 → 12348 [PSH, ACK] Seq=30 Ack=2 Win=65536 Len=5 TSval=2930223178 TSecr=2930223178
9	2.227431	127.0.0.1	127.0.0.1	TCP	71	12348 → 55774 [PSH, ACK] Seq=2 Ack=35 Win=65536 Len=5 TSval=2930223179 TSecr=2930223178
10	2.227441	127.0.0.1	127.0.0.1	TCP	71	55774 → 12348 [PSH, ACK] Seq=35 Ack=7 Win=65536 Len=5 TSval=2930223179 TSecr=2930223179
11	2.227464	127.0.0.1	127.0.0.1	TCP	71	12348 → 55774 [PSH, ACK] Seq=7 Ack=40 Win=65536 Len=5 TSval=2930223179 TSecr=2930223179
12	2.227630	127.0.0.1	127.0.0.1	TCP	71	55774 → 12348 [PSH, ACK] Seq=40 Ack=12 Win=65536 Len=5 TSval=2930223179 TSecr=2930223179
13	2.227658	127.0.0.1	127.0.0.1	TCP	71	12348 → 55774 [PSH, ACK] Seq=12 Ack=45 Win=65536 Len=5 TSval=2930223179 TSecr=2930223179
14	2.227662	127.0.0.1	127.0.0.1	TCP	71	55774 → 12348 [PSH, ACK] Seq=45 Ack=17 Win=65536 Len=5 TSval=2930223179 TSecr=2930223179
15	2.227673	127.0.0.1	127.0.0.1	TCP	71	12348 → 55774 [PSH, ACK] Seq=17 Ack=50 Win=65536 Len=5 TSval=2930223179 TSecr=2930223179
16	2.227695	127.0.0.1	127.0.0.1	TCP	71	55774 → 12348 [PSH, ACK] Seq=50 Ack=22 Win=65536 Len=5 TSval=2930223179 TSecr=2930223179
17	2.227723	127.0.0.1	127.0.0.1	TCP	71	12348 → 55774 [PSH, ACK] Seq=22 Ack=55 Win=65536 Len=5 TSval=2930223179 TSecr=2930223179
18	2.227729	127.0.0.1	127.0.0.1	TCP	71	55774 → 12348 [PSH, ACK] Seq=55 Ack=27 Win=65536 Len=5 TSval=2930223179 TSecr=2930223179
19	2.227747	127.0.0.1	127.0.0.1	TCP	71	12348 → 55774 [PSH, ACK] Seq=27 Ack=60 Win=65536 Len=5 TSval=2930223179 TSecr=2930223179
20	2.227766	127.0.0.1	127.0.0.1	TCP	71	55774 → 12348 [PSH, ACK] Seq=60 Ack=32 Win=65536 Len=5 TSval=2930223179 TSecr=2930223179
21	2.227796	127.0.0.1	127.0.0.1	TCP	71	12348 → 55774 [PSH, ACK] Seq=32 Ack=65 Win=65536 Len=5 TSval=2930223179 TSecr=2930223179
22	2.227804	127.0.0.1	127.0.0.1	TCP	71	55774 → 12348 [PSH, ACK] Seq=65 Ack=37 Win=65536 Len=5 TSval=2930223179 TSecr=2930223179
23	2.227823	127.0.0.1	127.0.0.1	TCP	71	12348 → 55774 [PSH, ACK] Seq=37 Ack=70 Win=65536 Len=5 TSval=2930223179 TSecr=2930223179
24	2.227841	127.0.0.1	127.0.0.1	TCP	71	55774 → 12348 [PSH, ACK] Seq=70 Ack=42 Win=65536 Len=5 TSval=2930223179 TSecr=2930223179
25	2.227857	127.0.0.1	127.0.0.1	TCP	71	12348 → 55774 [PSH, ACK] Seq=42 Ack=75 Win=65536 Len=5 TSval=2930223179 TSecr=2930223179
26	2.227876	127.0.0.1	127.0.0.1	TCP	71	55774 → 12348 [PSH, ACK] Seq=75 Ack=47 Win=65536 Len=5 TSval=2930223179 TSecr=2930223179
27	2.227899	127.0.0.1	127.0.0.1	TCP	71	12348 → 55774 [PSH, ACK] Seq=47 Ack=80 Win=65536 Len=5 TSval=2930223179 TSecr=2930223179
28	2.227918	127.0.0.1	127.0.0.1	TCP	71	55774 → 12348 [PSH, ACK] Seq=80 Ack=52 Win=65536 Len=5 TSval=2930223179 TSecr=2930223179
29	2.227935	127.0.0.1	127.0.0.1	TCP	71	12348 → 55774 [PSH, ACK] Seq=52 Ack=85 Win=65536 Len=5 TSval=2930223179 TSecr=2930223179
30	2.227950	127.0.0.1	127.0.0.1	TCP	71	55774 → 12348 [PSH, ACK] Seq=85 Ack=57 Win=65536 Len=5 TSval=2930223179 TSecr=2930223179
31	2.227976	127.0.0.1	127.0.0.1	TCP	71	12348 → 55774 [PSH, ACK] Seq=57 Ack=90 Win=65536 Len=5 TSval=2930223179 TSecr=2930223179
32	2.228005	127.0.0.1	127.0.0.1	TCP	66	55774 → 12348 [FIN, ACK] Seq=90 Ack=92 Win=65536 Len=0 TSval=2930223179 TSecr=2930223179
33	2.228054	127.0.0.1	127.0.0.1	TCP	66	12348 → 55774 [FIN, ACK] Seq=92 Ack=91 Win=65536 Len=0 TSval=2930223179 TSecr=2930223179
34	2.228100	127.0.0.1	127.0.0.1	TCP	66	55774 → 12348 [ACK] Seq=91 Ack=91 Win=65536 Len=0 TSval=2930223179 TSecr=2930223179

כמו שאפשר לראות מההקלטה , אנחנו מחלקים את ההודעה לשלחנו לחבילות קטנות יותר , אנחנו מקבלים עליהם ACK וממשיכים כך הלאה עד שמסיימים להעביר את כל ההודעה. **ההקלטה במקרה הזה נקראת q2.**

ננתח מקרה נוסף שבואו אנו מראים שאנחנו לא שולחים חבילה מתוך המחרוזת שחילקנו כלומר איבוד חבילה ואי קבלת חבילה ספציפית על מנת לעשות את זה , שינוי באופן זמני את הפונקציה הבאה :

```

24 def send_packets_in_window(client_socket, packets, recieved_ack_packets, current_start_windows_index, window_size):
25     for i in range(current_start_windows_index, min(current_start_windows_index + int(window_size), len(packets))):
26         if not recieved_ack_packets[i]:
27             print(f"Checking packet {i}")
28             if i == 2:
29                 print(f"Dropping packet {i} intentionally!")
30                 continue
31             print(f"Sending packet {i}: {packets[i]}")
32             client_socket.send(packets[i].encode())

```

כלומר אמרו שאם מספר הפאקטה שלנו 2 אז לדלג עליה ולא לשלוח אותה .

נשלח את הנתונים הבאים :

```

1 message:"This is a test message"
2 maximum_msg_size:5
3 window_size:4
4 timeout:1

```

נריך את השרת ולאחר מכן את הלקוח , כמו כן נפעיל את ההקלטה ב Wireshark

צד הלקוח :

```

Timeout reached while waiting for messages.
Checking packet 2
Dropping packet 2 intentionally!
Checking packet 3
Sending packet 3: M3:s
Checking packet 4
Sending packet 4: M4:a
Checking packet 5
Sending packet 5: M5:te
Timeout reached while waiting for messages.
Checking packet 2
Dropping packet 2 intentionally!
Checking packet 3
Sending packet 3: M3:s
Checking packet 4
Sending packet 4: M4:a
Checking packet 5
Sending packet 5: M5:te
Timeout reached while waiting for messages.
Checking packet 2
Dropping packet 2 intentionally!
Checking packet 3
Sending packet 3: M3:s
Checking packet 4
Sending packet 4: M4:a
Checking packet 5
Sending packet 5: M5:te
Timeout reached while waiting for messages.
Checking packet 2
Dropping packet 2 intentionally!
Checking packet 3

```

```

Connected to the server at localhost:12340
Choose one option:
1. Please provide data from Input
2. Please provide data from File
Enter your choice (1 or 2): 2
Received maximum message size: 5 bytes
Checking packet 0
Sending packet 0: M0:Th
Checking packet 1
Sending packet 1: M1:is
Checking packet 2
Dropping packet 2 intentionally!
Checking packet 3
Sending packet 3: M3:s
Received ACK0
Received ACK1
Timeout reached while waiting for messages.
Window is move to index 2
Checking packet 2
Dropping packet 2 intentionally!
Checking packet 3
Sending packet 3: M3:s
Checking packet 4
Sending packet 4: M4:a
Checking packet 5
Sending packet 5: M5:te
Timeout reached while waiting for messages.
Checking packet 2
Dropping packet 2 intentionally!
Checking packet 3
Sending packet 3: M3:s

```

צד השרת:

```

Sending ack message ACK1#
Server received message - M5:te
Out-of-order packet 5 received, still waiting for packet 2
Sending ack message ACK1#
Server received message - M3:s
Out-of-order packet 3 received, still waiting for packet 2
Sending ack message ACK1#
Server received message - M4:a
Out-of-order packet 4 received, still waiting for packet 2
Sending ack message ACK1#
Server received message - M5:te
Out-of-order packet 5 received, still waiting for packet 2
Sending ack message ACK1#
Server received message - M3:s
Out-of-order packet 3 received, still waiting for packet 2
Sending ack message ACK1#
Server received message - M4:a
Out-of-order packet 4 received, still waiting for packet 2
Sending ack message ACK1#
Server received message - M5:te
Out-of-order packet 5 received, still waiting for packet 2
Sending ack message ACK1#

```

```

Server started and (listening on $('', 12340))
Waiting for a client to connect...
Connection established with ('127.0.0.1', 42482)
Sending back maximum message size: 5
Server received message - M0:Th
Received expected packet 0
Sending ack message ACK0#
Server received message - M1:is
Received expected packet 1
Sending ack message ACK1#
Server received message - M3:s
Out-of-order packet 3 received, still waiting for packet 2
Sending ack message ACK1#
Server received message - M3:s
Out-of-order packet 3 received, still waiting for packet 2
Sending ack message ACK1#
Server received message - M4:a
Out-of-order packet 4 received, still waiting for packet 2
Sending ack message ACK1#
Server received message - M5:te
Out-of-order packet 5 received, still waiting for packet 2
Sending ack message ACK1#
Server received message - M3:s
Out-of-order packet 3 received, still waiting for packet 2
Sending ack message ACK1#
Server received message - M4:a
Out-of-order packet 4 received, still waiting for packet 2
Sending ack message ACK1#
Server received message - M5:te
Out-of-order packet 5 received, still waiting for packet 2
Sending ack message ACK1#
Server received message - M3:s

```


10.000000	127.0.0.1	127.0.0.1	TCP	74	42482	-	12340	[SYN, Seq=0 Win=65495 Len=0 MSS=65495 SACK_PERM=1 TSval=440114449 TSecr=0 WS=128	לחיצת היד המשותפת בקשה מהשרת את הגודל המקסימלי קבלת הגודל המקסימלי שליחת הפאקטה M0 שליחת הפאקטה M1
20.000013	127.0.0.1	127.0.0.1	TCP	74	12340	-	42482	[SYN, Seq=0 Ack=1 Win=65483 Len=0 MSS=65495 SACK_PERM=1 TSval=440114449 TSecr=440114449 WS=128	
30.000025	127.0.0.1	127.0.0.1	TCP	66	42482	-	12340	[ACK, Seq=1 Ack=1 Win=65536 Len=0 TSval=440114449 TSecr=440114449	
42.372773	127.0.0.1	127.0.0.1	TCP	95	42482	-	12340	[PSH, ACK, Seq=1 Ack=1 Win=65536 Len=29 TSval=440116822 TSecr=440114449	
52.372786	127.0.0.1	127.0.0.1	TCP	66	12340	-	42482	[ACK, Seq=1 Ack=30 Win=65536 Len=0 TSval=440116822 TSecr=440116822	
62.372995	127.0.0.1	127.0.0.1	TCP	67	12340	-	42482	[PSH, ACK, Seq=1 Ack=30 Win=65536 Len=1 TSval=440116822 TSecr=440116822	
72.372998	127.0.0.1	127.0.0.1	TCP	66	42482	-	12340	[ACK, Seq=30 Ack=2 Win=65536 Len=0 TSval=440116822 TSecr=440116822	
82.373118	127.0.0.1	127.0.0.1	TCP	71	12340	-	42482	[PSH, ACK, Seq=2 Ack=35 Win=65536 Len=5 TSval=440116822 TSecr=440116822	
102.373155	127.0.0.1	127.0.0.1	TCP	71	42482	-	12340	[PSH, ACK, Seq=35 Ack=7 Win=65536 Len=5 TSval=440116822 TSecr=440116822	
112.373163	127.0.0.1	127.0.0.1	TCP	71	12340	-	42482	[PSH, ACK, Seq=7 Ack=40 Win=65536 Len=5 TSval=440116822 TSecr=440116822	
122.373170	127.0.0.1	127.0.0.1	TCP	71	42482	-	12340	[PSH, ACK, Seq=40 Ack=12 Win=65536 Len=5 TSval=440116822 TSecr=440116822	
132.373179	127.0.0.1	127.0.0.1	TCP	71	12340	-	42482	[PSH, ACK, Seq=12 Ack=45 Win=65536 Len=5 TSval=440116822 TSecr=440116822	
142.463195	127.0.0.1	127.0.0.1	TCP	66	42482	-	12340	[ACK, Seq=45 Ack=17 Win=65536 Len=0 TSval=440116912 TSecr=440116822	
153.374864	127.0.0.1	127.0.0.1	TCP	71	42482	-	12340	[PSH, ACK, Seq=45 Ack=17 Win=65536 Len=5 TSval=440117824 TSecr=440116822	
163.374941	127.0.0.1	127.0.0.1	TCP	71	12340	-	42482	[PSH, ACK, Seq=17 Ack=50 Win=65536 Len=5 TSval=440117824 TSecr=440117824	
173.374946	127.0.0.1	127.0.0.1	TCP	66	42482	-	12340	[ACK, Seq=50 Ack=22 Win=65536 Len=0 TSval=440117824 TSecr=440117824	
183.374962	127.0.0.1	127.0.0.1	TCP	71	42482	-	12340	[PSH, ACK, Seq=50 Ack=22 Win=65536 Len=5 TSval=440117824 TSecr=440117824	
193.374995	127.0.0.1	127.0.0.1	TCP	71	12340	-	42482	[PSH, ACK, Seq=22 Ack=55 Win=65536 Len=5 TSval=440117824 TSecr=440117824	
203.374997	127.0.0.1	127.0.0.1	TCP	71	42482	-	12340	[PSH, ACK, Seq=55 Ack=27 Win=65536 Len=5 TSval=440117824 TSecr=440117824	
213.375011	127.0.0.1	127.0.0.1	TCP	71	12340	-	42482	[PSH, ACK, Seq=27 Ack=60 Win=65536 Len=5 TSval=440117824 TSecr=440117824	
223.415827	127.0.0.1	127.0.0.1	TCP	66	42482	-	12340	[ACK, Seq=60 Ack=32 Win=65536 Len=0 TSval=440117824 TSecr=440117824	
234.377655	127.0.0.1	127.0.0.1	TCP	71	42482	-	12340	[PSH, ACK, Seq=60 Ack=32 Win=65536 Len=5 TSval=440118827 TSecr=440117824	
244.377963	127.0.0.1	127.0.0.1	TCP	71	12340	-	42482	[PSH, ACK, Seq=32 Ack=65 Win=65536 Len=5 TSval=440118827 TSecr=440118827	
254.378028	127.0.0.1	127.0.0.1	TCP	76	42482	-	12340	[PSH, ACK, Seq=65 Ack=37 Win=65536 Len=10 TSval=440118827 TSecr=440118827	
264.378052	127.0.0.1	127.0.0.1	TCP	71	12340	-	42482	[ACK, Seq=37 Ack=75 Win=65536 Len=5 TSval=440118827 TSecr=440118827	
274.418936	127.0.0.1	127.0.0.1	TCP	66	42482	-	12340	[ACK, Seq=75 Ack=42 Win=65536 Len=0 TSval=440118868 TSecr=440118827	
284.418947	127.0.0.1	127.0.0.1	TCP	71	12340	-	42482	[PSH, ACK, Seq=42 Ack=75 Win=65536 Len=5 TSval=440118868 TSecr=440118868	
294.419811	127.0.0.1	127.0.0.1	TCP	66	42482	-	12340	[ACK, Seq=75 Ack=47 Win=65536 Len=0 TSval=440118868 TSecr=440118868	
305.380030	127.0.0.1	127.0.0.1	TCP	71	42482	-	12340	[PSH, ACK, Seq=75 Ack=47 Win=65536 Len=5 TSval=440119829 TSecr=440118868	
315.380324	127.0.0.1	127.0.0.1	TCP	71	12340	-	42482	[PSH, ACK, Seq=47 Ack=80 Win=65536 Len=5 TSval=440119829 TSecr=440119829	
325.380462	127.0.0.1	127.0.0.1	TCP	76	42482	-	12340	[PSH, ACK, Seq=80 Ack=52 Win=65536 Len=10 TSval=440119829 TSecr=440119829	
335.380483	127.0.0.1	127.0.0.1	TCP	71	12340	-	42482	[PSH, ACK, Seq=52 Ack=90 Win=65536 Len=5 TSval=440119829 TSecr=440119829	
345.426594	127.0.0.1	127.0.0.1	TCP	66	42482	-	12340	[ACK, Seq=90 Ack=57 Win=65536 Len=0 TSval=440119870 TSecr=440119829	
355.426666	127.0.0.1	127.0.0.1	TCP	71	12340	-	42482	[PSH, ACK, Seq=57 Ack=90 Win=65536 Len=5 TSval=440119870 TSecr=440119870	
365.426650	127.0.0.1	127.0.0.1	TCP	66	42482	-	12340	[ACK, Seq=90 Ack=62 Win=65536 Len=0 TSval=440119870 TSecr=440119870	
375.382169	127.0.0.1	127.0.0.1	TCP	71	42482	-	12340	[PSH, ACK, Seq=90 Ack=62 Win=65536 Len=5 TSval=440120831 TSecr=440119870	
385.382465	127.0.0.1	127.0.0.1	TCP	71	12340	-	42482	[PSH, ACK, Seq=62 Ack=95 Win=65536 Len=5 TSval=440120831 TSecr=440120831	
395.382473	127.0.0.1	127.0.0.1	TCP	76	42482	-	12340	[PSH, ACK, Seq=95 Ack=67 Win=65536 Len=10 TSval=440120831 TSecr=440120831	
405.382484	127.0.0.1	127.0.0.1	TCP	71	12340	-	42482	[PSH, ACK, Seq=67 Ack=105 Win=65536 Len=5 TSval=440120831 TSecr=440120831	
415.423132	127.0.0.1	127.0.0.1	TCP	66	42482	-	12340	[ACK, Seq=105 Ack=72 Win=65536 Len=0 TSval=440120872 TSecr=440120831	
425.423149	127.0.0.1	127.0.0.1	TCP	71	12340	-	42482	[PSH, ACK, Seq=72 Ack=105 Win=65536 Len=5 TSval=440120872 TSecr=440120872	
435.423168	127.0.0.1	127.0.0.1	TCP	66	42482	-	12340	[ACK, Seq=105 Ack=77 Win=65536 Len=0 TSval=440120872 TSecr=440120872	
447.382777	127.0.0.1	127.0.0.1	TCP	71	42482	-	12340	[PSH, ACK, Seq=105 Ack=77 Win=65536 Len=5 TSval=440121832 TSecr=440120872	
64.9.481961	127.0.0.1	127.0.0.1	TCP	71	42482	-	12340	[PSH, ACK, Seq=145 Ack=117 Win=65536 Len=5 TSval=440123931 TSecr=440123931	
65.9.481967	127.0.0.1	127.0.0.1	TCP	71	12340	-	42482	[PSH, ACK, Seq=117 Ack=150 Win=65536 Len=5 TSval=440123931 TSecr=440123931	
66.9.523204	127.0.0.1	127.0.0.1	TCP	66	42482	-	12340	[ACK, Seq=150 Ack=122 Win=65536 Len=0 TSval=440123972 TSecr=440123931	
67.10.48273	127.0.0.1	127.0.0.1	TCP	71	42482	-	12340	[PSH, ACK, Seq=150 Ack=122 Win=65536 Len=5 TSval=440124932 TSecr=440123931	
68.10.48281	127.0.0.1	127.0.0.1	TCP	71	12340	-	42482	[PSH, ACK, Seq=122 Ack=155 Win=65536 Len=5 TSval=440124932 TSecr=440124932	
69.10.48282	127.0.0.1	127.0.0.1	TCP	66	42482	-	12340	[ACK, Seq=155 Ack=127 Win=65536 Len=0 TSval=440124932 TSecr=440124932	
70.10.48283	127.0.0.1	127.0.0.1	TCP	71	42482	-	12340	[PSH, ACK, Seq=155 Ack=127 Win=65536 Len=5 TSval=440124932 TSecr=440124932	
71.10.48284	127.0.0.1	127.0.0.1	TCP	71	12340	-	42482	[PSH, ACK, Seq=127 Ack=160 Win=65536 Len=5 TSval=440124932 TSecr=440124932	
72.10.48285	127.0.0.1	127.0.0.1	TCP	71	42482	-	12340	[PSH, ACK, Seq=160 Ack=132 Win=65536 Len=5 TSval=440124932 TSecr=440124932	
73.10.48285	127.0.0.1	127.0.0.1	TCP	71	12340	-	42482	[PSH, ACK, Seq=132 Ack=165 Win=65536 Len=5 TSval=440124932 TSecr=440124932	
74.10.52362	127.0.0.1	127.0.0.1	TCP	66	42482	-	12340	[ACK, Seq=165 Ack=137 Win=65536 Len=0 TSval=440124973 TSecr=440124932	
75.11.48449	127.0.0.1	127.0.0.1	TCP	71	42482	-	12340	[PSH, ACK, Seq=165 Ack=137 Win=65536 Len=5 TSval=440125933 TSecr=440124932	
76.11.48456	127.0.0.1	127.0.0.1	TCP	71	12340	-	42482	[PSH, ACK, Seq=137 Ack=165 Win=65536 Len=5 TSval=440125933 TSecr=440125933	
77.11.48457	127.0.0.1	127.0.0.1	TCP	66	42482	-	12340	[ACK, Seq=170 Ack=142 Win=65536 Len=0 TSval=440125934 TSecr=440125934	
78.11.48458	127.0.0.1	127.0.0.1	TCP	71	42482	-	12340	[PSH, ACK, Seq=170 Ack=142 Win=65536 Len=5 TSval=440125934 TSecr=440125934	
79.11.48459	127.0.0.1	127.0.0.1	TCP	71	12340	-	42482	[PSH, ACK, Seq=142 Ack=175 Win=65536 Len=5 TSval=440125934 TSecr=440125934	
80.11.48459	127.0.0.1	127.0.0.1	TCP	71	42482	-	12340	[PSH, ACK, Seq=175 Ack=147 Win=65536 Len=5 TSval=440125934 TSecr=440125934	
81.11.48460	127.0.0.1	127.0.0.1	TCP	71	12340	-	42482	[PSH, ACK, Seq=147 Ack=180 Win=65536 Len=5 TSval=440125934 TSecr=440125934	
82.11.52483	127.0.0.1	127.0.0.1	TCP	66	42482	-	12340	[ACK, Seq=180 Ack=152 Win=65536 Len=0 TSval=440125974 TSecr=440125934	
83.12.48767	127.0.0.1	127.0.0.1	TCP	71	42482	-	12340	[PSH, ACK, Seq=180 Ack=152 Win=65536 Len=5 TSval=440126937 TSecr=440125934	
84.12.48773	127.0.0.1	127.0.0.1	TCP	71	12340	-	42482	[PSH, ACK, Seq=152 Ack=185 Win=65536 Len=5 TSval=440126937 TSecr=440126937	
85.12.48774	127.0.0.1	127.0.0.1	TCP	66	42482	-	12340	[ACK, Seq=185 Ack=157 Win=65536 Len=0 TSval=440126937 TSecr=440126937	
86.12.48777	127.0.0.1	127.0.0.1	TCP	71	42482	-	12340	[PSH, ACK, Seq=185 Ack=157 Win=65536 Len=5 TSval=440126937 TSecr=440126937	
87.12.48778	127.0.0.1	127.0.0.1	TCP	71	12340	-	42482	[PSH, ACK, Seq=157 Ack=190 Win=65536 Len=5 TSval=440126937 TSecr=440126937	
88.12.48778	127.0.0.1	127.0.0.1	TCP	71	42482	-	12340	[PSH, ACK, Seq=190 Ack=162 Win=65536 Len=5 TSval=440126937 TSecr=440126937	
89.12.48779	127.0.0.1	127.0.0.1	TCP	71	12340	-	42482	[PSH, ACK, Seq=162 Ack=195 Win=65536 Len=5 TSval=440126937 TSecr=440126937	
90.12.58081	127.0.0.1	127.0.0.1	TCP	66	42482	-	12340	[ACK, Seq=195 Ack=167 Win=65536 Len=0 TSval=440127035 TSecr=440126937	
91.13.49390	127.0.0.1	127.0.0.1	TCP	71	42482	-	12340	[PSH, ACK, Seq=195 Ack=167 Win=65536 Len=5 TSval=440127943 TSecr=440126937	
92.13.49497	127.0.0.1	127.0.0.1	TCP	71	12340	-	42482	[PSH, ACK, Seq=167 Ack=200 Win=65536 Len=5 TSval=440127944 TSecr=440127943	
93.13.49497	127.0.0.1	127.0.0.1	TCP	66	42482	-	12340	[ACK, Seq=200 Ack=172 Win=65536 Len=0 TSval=440127944 TSecr=440127944	
94.13.49486	127.0.0.1	127.0.0.1	TCP	71	42482	-	12340	[PSH, ACK, Seq=200 Ack=172 Win=65536 Len=5 TSval=440127944 TSecr=440127944	

```
def handle_client(client_socket):
    usage

    if packet_number == packet_expected_from_client:
        print(f"Received expected packet {packet_number}")
        packet_expected_from_client += 1
        while packet_expected_from_client < len(packets_received) and packets_received[
            packet_expected_from_client] is not None:
            packet_expected_from_client += 1
    else:
        print(
            f"Out-of-order packet {packet_number} received, still waiting for packet {packet_expected_from_client}")

    ack_message = f"ACK{packet_expected_from_client - 1}"
    ack_message = ack_message.ljust(max_size_number, _fillchar: '#')
    print(f"Sending ack-message {ack_message}")
    time_out = 1
    time.sleep(time_out)
    client_socket.send(ack_message.encode())
except Exception as e:
    print(f"Error handling client: {e}")
```

זה יגרום לשרת להתעכב בשליחת האק של ללקוח ובכך לגרום לשליחה חוזרת של פאקטות
שוב נשלח את הנתונים הבאים :

```
1 message:"This is a test message"
2 maximum_msg_size:5
3 window_size:4
4 timeout:1
```

נפעיל את השרת את הלקוח וכמו כן נפעיל את הקלטת Wireshark על מנת לנתח את המידע
צד הלקוח :

```
Choose one option:
1. Please provide data from Input
2. Please provide data from File
Enter your choice (1 or 2): 2
Received maximum message size: 5 bytes

Sending packet 0: M0:Th
Sending packet 1: M1:is
Sending packet 2: M2: i
Sending packet 3: M3:s
Timeout reached while waiting for messages.
Sending packet 0: M0:Th
Sending packet 1: M1:is
Sending packet 2: M2: i
Sending packet 3: M3:s
Timeout reached while waiting for messages.
Sending packet 0: M0:Th
Sending packet 1: M1:is
Sending packet 2: M2: i
Sending packet 3: M3:s
Timeout reached while waiting for messages.
Sending packet 0: M0:Th
Sending packet 1: M1:is
Sending packet 2: M2: i
Sending packet 3: M3:s
```

קבלת Maxsize

שליחה של החלון הראשון

הודעת TIMEOUT

שליחה חוזרת עבור החלון הזה עקב אי עמידה בזמן המוגדר

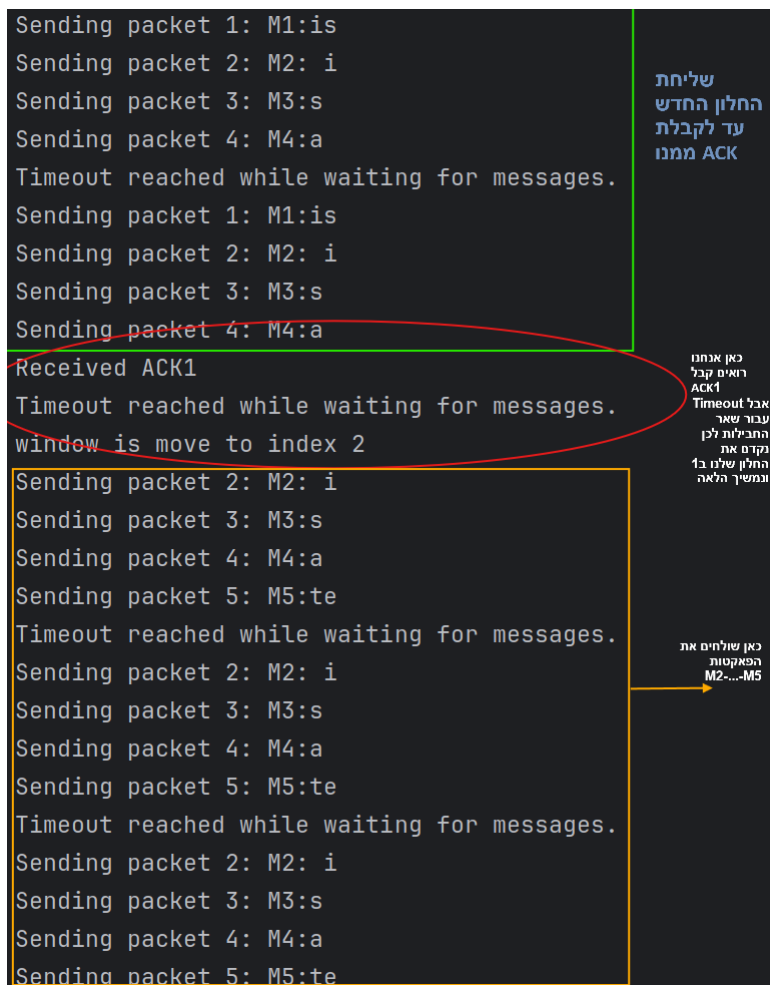
```

Timeout reached while waiting for messages.
Sending packet 0: M0:Th
Sending packet 1: M1:is
Sending packet 2: M2: i
Sending packet 3: M3:s
Received ACK0
Timeout reached while waiting for messages.
window is move to index 1
Sending packet 1: M1:is
Sending packet 2: M2: i
Sending packet 3: M3:s
Sending packet 4: M4:a
Timeout reached while waiting for messages.
Sending packet 1: M1:is
Sending packet 2: M2: i
Sending packet 3: M3:s
Sending packet 4: M4:a
Timeout reached while waiting for messages.
Sending packet 1: M1:is
Sending packet 2: M2: i
Sending packet 3: M3:s
Sending packet 4: M4:a
Timeout reached while waiting for messages.
Sending packet 1: M1:is
Sending packet 2: M2: i
Sending packet 3: M3:s
Sending packet 4: M4:a
Timeout reached while waiting for messages.
Sending packet 1: M1:is
Sending packet 2: M2: i
Sending packet 3: M3:s
Sending packet 4: M4:a

```

שליחת החלון
שוב וקבלת
ACK0

קידום
החלון
שלנו ב-1
ושליחת
M1....-M4



וזה הפלט עד שמגיעים לשליחת כל ההודעה שהלקוח רוצה להעביר (זה ארוך לצלם את כל זה אבל כעיקרון זה מה שקורא ואפשר להראות את זה גם בהקלטות בWireshark)

לבסוף נראה כך

```

General timeout reached.
timeout is occurred
Sending packet 11: M11:e
General timeout reached.
timeout is occurred
Sending packet 11: M11:e
Timeout reached while waiting for messages.
Sending packet 11: M11:e
General timeout reached.
timeout is occurred
Sending packet 11: M11:e
Timeout reached while waiting for messages.
Sending packet 11: M11:e
Timeout reached while waiting for messages.
Sending packet 11: M11:e
General timeout reached.
timeout is occurred
Sending packet 11: M11:e
Received ACK11
General timeout reached.
timeout is occurred

```

אנחנו שולחים את כל
החלקים

כאן אנו מקבלים את
כל ההודעה ומקבלים
אישור על כך

```

window is move to index 12
Finishing sending all messages + gets all acks back

```

בצד השרת נראה זאת כך :


```

/home/Ron/PycharmProjects/PythonProject/.venv/bin/python /home/Ron/PycharmProjects/PythonProject/server.py
Server started and listening on $(' ', 12341)
Waiting for a client to connect...
Connection established with ('127.0.0.1', 56826)
Sending back maximum message size: 5
Server received message - M0:Th
Received expected packet 0
Sending ack message ACK0#
Server received message - M1:is
Received expected packet 1
Sending ack message ACK1#
Server received message - M2: i
Received expected packet 2
Sending ack message ACK2#
Server received message - M3:s
Received expected packet 3
Sending ack message ACK3#
Server received message - M1:is
Out-of-order packet 1 received, still waiting for packet 4
Sending ack message ACK3#
Server received message - M2: i
Out-of-order packet 2 received, still waiting for packet 4
Sending ack message ACK3#
Server received message - M3:s
Out-of-order packet 3 received, still waiting for packet 4
Sending ack message ACK3#
Server received message - M4:a
Received expected packet 4
Sending ack message ACK4#
Server received message - M2: i
Out-of-order packet 2 received, still waiting for packet 5
Sending ack message ACK4#

```

וכן הלאה עבור שאר הפאקטות (אפשר לראות את זה בWireshark).

No.	Time	Source	Destination	Protocol	Length	Info
1	0.000000	127.0.0.1	127.0.0.1	TCP	74	56826 → 12341 [SYN] Seq=0 Win=65495 Len=0 MSS=65495 SACK_PERM=1 TSval=447308286 TSecr=0 WS=128
2	0.000018	127.0.0.1	127.0.0.1	TCP	74	12341 → 56826 [SYN, ACK] Seq=0 Ack=1 Win=65483 Len=0 MSS=65495 SACK_PERM=1 TSval=447308286 TSecr=447308285 WS=128
3	0.000032	127.0.0.1	127.0.0.1	TCP	66	56826 → 12341 [ACK] Seq=1 Ack=1 Win=65536 Len=0 TSval=447308286 TSecr=447308286
4	2.095320	127.0.0.1	127.0.0.1	TCP	95	56826 → 12341 [PSH, ACK] Seq=1 Ack=1 Win=65536 Len=29 TSval=447310381 TSecr=447308286
5	2.095411	127.0.0.1	127.0.0.1	TCP	66	12341 → 56826 [ACK] Seq=1 Ack=30 Win=65536 Len=0 TSval=447310381 TSecr=447310381
6	2.096922	127.0.0.1	127.0.0.1	TCP	67	12341 → 56826 [PSH, ACK] Seq=1 Ack=30 Win=65536 Len=1 TSval=447310382 TSecr=447310381
7	2.096962	127.0.0.1	127.0.0.1	TCP	66	56826 → 12341 [ACK] Seq=30 Ack=2 Win=65536 Len=0 TSval=447310382 TSecr=447310382
8	2.097455	127.0.0.1	127.0.0.1	TCP	71	56826 → 12341 [PSH, ACK] Seq=30 Ack=2 Win=65536 Len=5 TSval=447310383 TSecr=447310382
9	2.160457	127.0.0.1	127.0.0.1	TCP	66	12341 → 56826 [ACK] Seq=2 Ack=35 Win=65536 Len=0 TSval=447310446 TSecr=447310383
10	2.160474	127.0.0.1	127.0.0.1	TCP	81	56826 → 12341 [PSH, ACK] Seq=35 Ack=2 Win=65536 Len=15 TSval=447310446 TSecr=447310446
11	2.160483	127.0.0.1	127.0.0.1	TCP	66	12341 → 56826 [ACK] Seq=2 Ack=50 Win=65536 Len=0 TSval=447310446 TSecr=447310446
12	3.099066	127.0.0.1	127.0.0.1	TCP	71	12341 → 56826 [PSH, ACK] Seq=2 Ack=50 Win=65536 Len=5 TSval=447311385 TSecr=447310446
13	3.101027	127.0.0.1	127.0.0.1	TCP	71	56826 → 12341 [PSH, ACK] Seq=50 Ack=7 Win=65536 Len=5 TSval=447311387 TSecr=447311385
14	3.101035	127.0.0.1	127.0.0.1	TCP	66	12341 → 56826 [ACK] Seq=7 Ack=55 Win=65536 Len=0 TSval=447311387 TSecr=447311387
15	3.101044	127.0.0.1	127.0.0.1	TCP	71	56826 → 12341 [PSH, ACK] Seq=55 Ack=7 Win=65536 Len=5 TSval=447311387 TSecr=447311387
16	3.101045	127.0.0.1	127.0.0.1	TCP	66	12341 → 56826 [ACK] Seq=7 Ack=60 Win=65536 Len=0 TSval=447311387 TSecr=447311387
17	3.101050	127.0.0.1	127.0.0.1	TCP	71	56826 → 12341 [PSH, ACK] Seq=60 Ack=7 Win=65536 Len=5 TSval=447311387 TSecr=447311387
18	3.101051	127.0.0.1	127.0.0.1	TCP	66	12341 → 56826 [ACK] Seq=7 Ack=65 Win=65536 Len=0 TSval=447311387 TSecr=447311387
19	3.101055	127.0.0.1	127.0.0.1	TCP	71	56826 → 12341 [PSH, ACK] Seq=65 Ack=7 Win=65536 Len=5 TSval=447311387 TSecr=447311387
20	3.101056	127.0.0.1	127.0.0.1	TCP	66	12341 → 56826 [ACK] Seq=7 Ack=70 Win=65536 Len=0 TSval=447311387 TSecr=447311387
21	4.100708	127.0.0.1	127.0.0.1	TCP	71	12341 → 56826 [PSH, ACK] Seq=7 Ack=70 Win=65536 Len=5 TSval=447312386 TSecr=447311387
22	4.101164	127.0.0.1	127.0.0.1	TCP	71	56826 → 12341 [PSH, ACK] Seq=70 Ack=12 Win=65536 Len=5 TSval=447312387 TSecr=447312386
23	4.101170	127.0.0.1	127.0.0.1	TCP	66	12341 → 56826 [ACK] Seq=12 Ack=75 Win=65536 Len=0 TSval=447312387 TSecr=447312387
24	4.101177	127.0.0.1	127.0.0.1	TCP	71	56826 → 12341 [PSH, ACK] Seq=75 Ack=12 Win=65536 Len=5 TSval=447312387 TSecr=447312387
25	4.101177	127.0.0.1	127.0.0.1	TCP	66	12341 → 56826 [ACK] Seq=12 Ack=88 Win=65536 Len=0 TSval=447312387 TSecr=447312387
26	4.101181	127.0.0.1	127.0.0.1	TCP	71	56826 → 12341 [PSH, ACK] Seq=88 Ack=12 Win=65536 Len=5 TSval=447312387 TSecr=447312387
27	4.101182	127.0.0.1	127.0.0.1	TCP	66	12341 → 56826 [ACK] Seq=12 Ack=85 Win=65536 Len=0 TSval=447312387 TSecr=447312387
28	4.101184	127.0.0.1	127.0.0.1	TCP	71	56826 → 12341 [PSH, ACK] Seq=85 Ack=12 Win=65536 Len=5 TSval=447312387 TSecr=447312387
29	4.101185	127.0.0.1	127.0.0.1	TCP	66	12341 → 56826 [ACK] Seq=12 Ack=90 Win=65536 Len=0 TSval=447312387 TSecr=447312387
30	5.103442	127.0.0.1	127.0.0.1	TCP	71	12341 → 56826 [PSH, ACK] Seq=12 Ack=90 Win=65536 Len=5 TSval=447313389 TSecr=447312387
31	5.103624	127.0.0.1	127.0.0.1	TCP	71	56826 → 12341 [PSH, ACK] Seq=90 Ack=17 Win=65536 Len=5 TSval=447313389 TSecr=447313389
32	5.103632	127.0.0.1	127.0.0.1	TCP	66	12341 → 56826 [ACK] Seq=17 Ack=95 Win=65536 Len=0 TSval=447313389 TSecr=447313389
33	5.103633	127.0.0.1	127.0.0.1	TCP	71	56826 → 12341 [PSH, ACK] Seq=95 Ack=17 Win=65536 Len=5 TSval=447313389 TSecr=447313389
34	5.103640	127.0.0.1	127.0.0.1	TCP	66	12341 → 56826 [ACK] Seq=17 Ack=100 Win=65536 Len=0 TSval=447313389 TSecr=447313389
35	5.103643	127.0.0.1	127.0.0.1	TCP	71	56826 → 12341 [PSH, ACK] Seq=100 Ack=17 Win=65536 Len=5 TSval=447313389 TSecr=447313389
36	5.103644	127.0.0.1	127.0.0.1	TCP	66	12341 → 56826 [ACK] Seq=17 Ack=105 Win=65536 Len=0 TSval=447313389 TSecr=447313389
37	5.103647	127.0.0.1	127.0.0.1	TCP	71	56826 → 12341 [PSH, ACK] Seq=105 Ack=17 Win=65536 Len=5 TSval=447313389 TSecr=447313389
38	5.103648	127.0.0.1	127.0.0.1	TCP	66	12341 → 56826 [ACK] Seq=17 Ack=110 Win=65536 Len=0 TSval=447313389 TSecr=447313389
39	6.172782	127.0.0.1	127.0.0.1	TCP	71	56826 → 12341 [PSH, ACK] Seq=110 Ack=17 Win=65536 Len=5 TSval=447314458 TSecr=447313389
40	6.172836	127.0.0.1	127.0.0.1	TCP	71	12341 → 56826 [PSH, ACK] Seq=17 Ack=110 Win=65536 Len=5 TSval=447314458 TSecr=447313389
41	6.172846	127.0.0.1	127.0.0.1	TCP	66	12341 → 56826 [ACK] Seq=22 Ack=115 Win=65536 Len=0 TSval=447314458 TSecr=447314458
42	6.172799	127.0.0.1	127.0.0.1	TCP	71	56826 → 12341 [PSH, ACK] Seq=115 Ack=22 Win=65536 Len=5 TSval=447314458 TSecr=447314458
43	6.172802	127.0.0.1	127.0.0.1	TCP	66	12341 → 56826 [ACK] Seq=22 Ack=120 Win=65536 Len=0 TSval=447314458 TSecr=447314458
44	6.172806	127.0.0.1	127.0.0.1	TCP	71	56826 → 12341 [PSH, ACK] Seq=120 Ack=22 Win=65536 Len=5 TSval=447314458 TSecr=447314458
45	6.172807	127.0.0.1	127.0.0.1	TCP	66	12341 → 56826 [ACK] Seq=22 Ack=125 Win=65536 Len=0 TSval=447314458 TSecr=447314458
46	6.172810	127.0.0.1	127.0.0.1	TCP	71	56826 → 12341 [PSH, ACK] Seq=125 Ack=22 Win=65536 Len=5 TSval=447314458 TSecr=447314458

לחיצת יד משולשת

גודל ההודעה

שליחת M0

כאן אנחנו רואים שליחה של M0...M3 וקבלת ACK של חלקם

בהקלטה המצורפת אפשר לראות את הכול ואת השליחה וקבלת הACK של שאר הפאקטות .

649 98.78227	127.0.0.1	127.0.0.1	TCP	66	56826 → 12341 [ACK] Seq=1440 Ack=482 Win=65536 Len=0 TSval=447407068 TSecr=447407068
650 98.78240	127.0.0.1	127.0.0.1	TCP	71	56826 → 12341 [PSH, ACK] Seq=1440 Ack=482 Win=65536 Len=5 TSval=447407068 TSecr=447407068
651 98.82310	127.0.0.1	127.0.0.1	TCP	66	12341 → 56826 [ACK] Seq=482 Ack=1445 Win=65536 Len=0 TSval=447407109 TSecr=447407068
652 99.78345	127.0.0.1	127.0.0.1	TCP	71	12341 → 56826 [PSH, ACK] Seq=482 Ack=1445 Win=65536 Len=5 TSval=447408069 TSecr=447407068
653 99.78333	127.0.0.1	127.0.0.1	TCP	71	56826 → 12341 [PSH, ACK] Seq=1445 Ack=487 Win=65536 Len=5 TSval=447408069 TSecr=447408069
654 99.78334	127.0.0.1	127.0.0.1	TCP	66	12341 → 56826 [ACK] Seq=22 Ack=115 Win=65536 Len=0 TSval=447408069 TSecr=447408069
655 100.78408	127.0.0.1	127.0.0.1	TCP	71	56826 → 12341 [PSH, ACK] Seq=1450 Ack=487 Win=65536 Len=5 TSval=447409069 TSecr=447408069
656 100.78408	127.0.0.1	127.0.0.1	TCP	66	12341 → 56826 [ACK] Seq=487 Ack=1455 Win=65536 Len=0 TSval=447409070 TSecr=447409069
657 100.7843	127.0.0.1	127.0.0.1	TCP	71	12341 → 56826 [PSH, ACK] Seq=487 Ack=1455 Win=65536 Len=5 TSval=447409070 TSecr=447409069
658 100.8250	127.0.0.1	127.0.0.1	TCP	66	12341 → 56826 [ACK] Seq=1455 Ack=492 Win=65536 Len=0 TSval=447409111 TSecr=447409070
659 101.7851	127.0.0.1	127.0.0.1	TCP	71	12341 → 56826 [PSH, ACK] Seq=492 Ack=1455 Win=65536 Len=5 TSval=447410071 TSecr=447409111
660 101.7852	127.0.0.1	127.0.0.1	TCP	66	12341 → 56826 [ACK] Seq=1455 Ack=487 Win=65536 Len=0 TSval=447410071 TSecr=447410071
661 101.7853	127.0.0.1	127.0.0.1	TCP	71	56826 → 12341 [PSH, ACK] Seq=1455 Ack=487 Win=65536 Len=5 TSval=447410071 TSecr=447410071
662 101.8262	127.0.0.1	127.0.0.1	TCP	66	12341 → 56826 [ACK] Seq=497 Ack=1480 Win=65536 Len=0 TSval=447410112 TSecr=447410071
663 102.7862	127.0.0.1	127.0.0.1	TCP	71	12341 → 56826 [PSH, ACK] Seq=497 Ack=1480 Win=65536 Len=5 TSval=447411072 TSecr=447410071
664 102.7863	127.0.0.1	127.0.0.1	TCP	66	56826 → 12341 [FIN, ACK] Seq=1400 Ack=502 Win=65536 Len=0 TSval=447411072 TSecr=447411072
665 102.8371	127.0.0.1	127.0.0.1	TCP	66	12341 → 56826 [ACK] Seq=502 Ack=1481 Win=65536 Len=0 TSval=447411123 TSecr=447411072
666 103.7878	127.0.0.1	127.0.0.1	TCP	71	12341 → 56826 [PSH, ACK] Seq=502 Ack=1481 Win=65536 Len=5 TSval=447412073 TSecr=447411072

העברת כל ההודעות וסגירת הקשר

להקלטה Wireshark נקרא q4 .

כמו כן נחזיר את הקוד למצבו המקורי על מנת למנוע שגיאות בבדיקה , כמו שנאמר השינוי הוא על מנת לבחון ולנתח את המקרה של שידור חוזר .

כעת נדמה בוקד , את המקרה של שידור חוזר של הפאקטה בכך שהיא לא מקבלת ACK בזמן המוגדר לה , כלומר בדוגמה זו ספציפית נדמה מקרה בו הפאקטות מספר 2 ומספר 10 לא מקבלות ACK בזמן הדרוש ולכן שולחים אותן שוב. נתחיל בהפעלת השרת ולאחר מכן הפעלת הלקוח כמו כן נפעיל את האזנה בWireshark

השינויים שביצענו הם בצד השרת משום שהוא אחראי על שליחת הACK עבור חבילות מתקבלות , בפונקציה הנקראת handle_client

להלן השינוי :

```
def handle_client(client_socket):
    usage = 1
    print(f"Received expected packet {packet_number}")
    packet_expected_from_client += 1
    while packet_expected_from_client < len(packets_received) and packets_received[
        packet_expected_from_client] is not None:
        packet_expected_from_client += 1
    else:
        print(
            f"Out-of-order packet {packet_number} received, still waiting for packet {packet_expected_from_client}")
    # Introduce a delay if packet number 2 is received
    if packet_number == 2 or packet_number == 10:
        print("Received packet 2 or 10, delaying ACK by 2 seconds...")
        time.sleep(2)
    ack_message = f"ACK{packet_expected_from_client - 1}"
    ack_message = ack_message.ljust(max_size_number, _fillchar, '#')
    print(f"Sending ack message {ack_message}")
    client_socket.send(ack_message.encode())
except Exception as e:
    print(f"Error handling client: {e}")
```

השינוי הזה אומר שכאשר פאקטות מספר 2 ו-10 מתקבלות נעשה עיכוב של 2 שניות עבור שליחת הACK בעזרת sleep. אנחנו עושים זאת כדי לדמה את המקרה בו החבילה ששלחנו לא מקבלת ACK ולכן נצטרך לשדר אותה מחדש.

כמו כן הנתונים ששלחנו הם :

```
Server.py  input.txt  Client.py
1 message:"This is a test message"
2 maximum_msg_size:5
3 window_size:4
4 timeout:2
5
6
```

צד הלקוח :

```
/home/ron/PycharmProjects/PythonProject/.venv/bin/python /home/ron/PycharmProjects/PythonProject/Client.py
Connected to the server at localhost:12348
Choose one option:
1. Please provide data from Input
2. Please provide data from File
Enter your choice (1 or 2): 2
Received maximum message size: 5 bytes
Sending packet 0: M0:Th
Sending packet 1: M1:is
Sending packet 2: M2: i
Sending packet 3: M3:s
Received ACK0
Received ACK1
Received ACK2
General timeout reached.
timeout is occurred
window is move to index 3
Sending packet 3: M3:s
Sending packet 4: M4:a
Sending packet 5: M5:te
Sending packet 6: M6:st
Received ACK3
Received ACK4
Received ACK5
Received ACK6
window is move to index 7
```

כאן אנחנו בוחרים באיזה אופציה לקבל את maxsize

כאן אנחנו שולחים את הפאקטות בין M0...-M3 ואים קבלת ACK 0, 1, 2 ומקבלים timeout ביז את החלון שלנו באוד נמשך שוב את M3

כאן אפשר לראות שידור חוזר של M3 ושליחה נוספת של חבילות וקבלת ACK עליהם וכמו כן הדות החלון

```
Sending packet 7: M7: m
Sending packet 8: M8:es
Sending packet 9: M9:sa
Sending packet 10: M10:g
Received ACK7
Received ACK8
Received ACK9
Received ACK10
window is move to index 11
Sending packet 11: M11:e
Received ACK11
Timeout reached while waiting for messages.
window is move to index 12
Finishing sending all messages + gets all acks back
```

שליחת הפאקטות הבאות בתור (כאן אפשר לראות שהוספת העיכוב לא גורם לשליחה חוזרת של הפאקטה)

שליחה של הפאקטה האחרונה

קיבלנו את כל ההודעה וגם את כל הACK

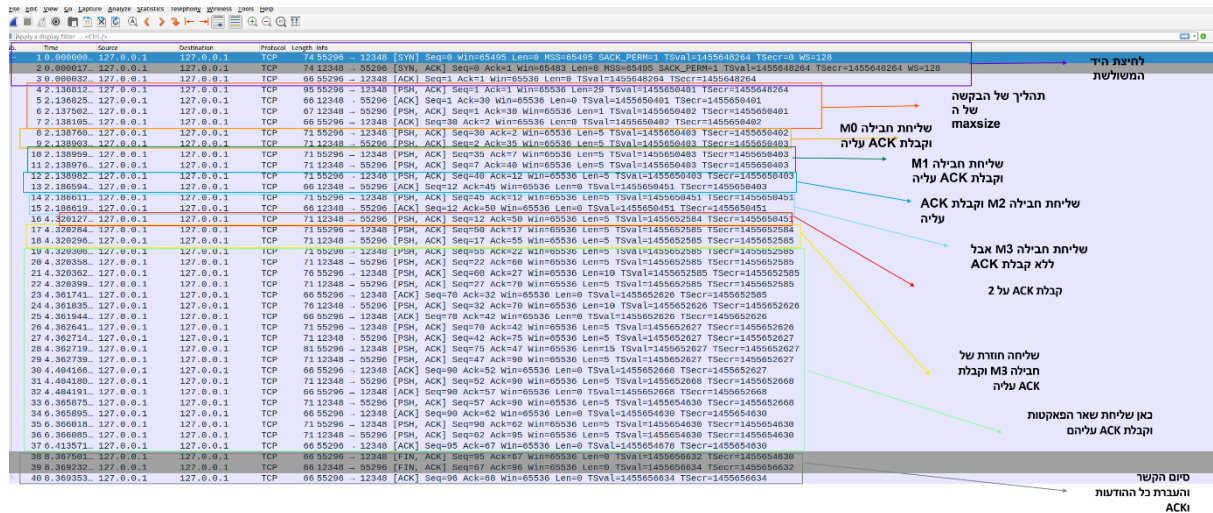
צד השרת :

```
/home/ron/PycharmProjects/PythonProject/.venv/bin/python /home/ron/PycharmProjects/PythonProject/Server.py
Server started and listening on $('', 12348)
Waiting for a client to connect...
Connection established with ('127.0.0.1', 55296)
Sending back maximum message size: 5
Server received message - M0:Th
Received expected packet 0
Sending ack message ACK0#
Server received message - M1:is
Received expected packet 1
Sending ack message ACK1#
Server received message - M2: i
Received expected packet 2
Received packet 2 or 10, delaying ACK by 2 seconds...
Sending ack message ACK2#
Server received message - M3:s
Received expected packet 3
Sending ack message ACK3#
Server received message - M3:s
Out-of-order packet 3 received, still waiting for packet 4
Sending ack message ACK3#
Server received message - M4:a
Received expected packet 4
Sending ack message ACK4#
Server received message - M5:te
Received expected packet 5
Sending ack message ACK5#
```

```
Server received message - M5:te
Received expected packet 5
Sending ack message ACK5#
Server received message - M6:st
Received expected packet 6
Sending ack message ACK6#
Server received message - M7: m
Received expected packet 7
Sending ack message ACK7#
Server received message - M8:es
Received expected packet 8
Sending ack message ACK8#
Server received message - M9:sa
Received expected packet 9
Sending ack message ACK9#
Server received message - M10:g
Received expected packet 10
Received packet 2 or 10, delaying ACK by 2 seconds...
Sending ack message ACK10
Server received message - M11:e
Received expected packet 11
Sending ack message ACK11
Client is finished, closing socket
Waiting for a client to connect...
```

בצד השרת אנו רואים את כל הACK עבור כל החבילות ששלחנו , ואת העיכובים של הפאקטות 1,2 .

כעת ננתח את התעבורה שעוברת בWireshark:



אפשר לראות בהקלטה את השידור החוזר של החבילה עקב timeout .

להקלטה הזו נקרא q5.

כמו שנאמר זה רק על מנת לדמות סטואציה ספציפית ולכן נחזיר את הקוד שלנו לקוד המקורי .

למען הסרת הספק :

כל ההקלטות הן על קריאה מקובץ ולא קריאה מINPUT מהסיבה הפשוטה שיהיה יותר קל להסביר.

נוסיף עוד הקלטה המראה באופן בסיסי את העברה של ההודעה כאשר אנחנו קולטים אותה מהקונסול

הפעלת השרת :

```
Run Server Client
/home/ron/PycharmProjects/PythonProject/.venv/bin/python /home/ron/PycharmProjects/PythonProject/Server.py
Server started and listening on $('', 12348)
Waiting for a client to connect...
```

הפעלת הלקוח ובחירה באופציה מספר 1:

```
Run Server Client
/home/ron/PycharmProjects/PythonProject/.venv/bin/python /home/ron/PycharmProjects/PythonProject/Client.py
Connected to the server at localhost:12348
Choose one option:
1. Please provide data from Input
2. Please provide data from File
Enter your choice (1 or 2): 1
```

הלקוח מחכה שהשרת יחזיר לו את ההודעה המקסימלית שהוא יכול לקבל נניח כנניס 5


```
Run Server Client
/home/ron/PycharmProjects/PythonProject/.venv/bin/python /home/ron/PycharmProjects/PythonProject/Server.py
Server started and listening on ('', 12348)
Waiting for a client to connect...
Connection established with ('127.0.0.1', 58100)
Enter the maximum message size (in bytes): 5
Sending back maximum message size: 5
```

נחזור ללקוח ונראה שהוא קיבל את גודל ההודעה המקסימלי וכמו כן נכניס את הנתונים הבאים :

```
Received maximum message size: 5 bytes
Enter the message: This is a test message
Enter the window size (number of messages): 4
Enter the timeout (in seconds): 3
```

לאחר מכן נעביר את ההודעה

צד הלקוח :

```
Sending packet 0: M0:Th
Sending packet 1: M1:is
Sending packet 2: M2: i
Sending packet 3: M3:s
Received ACK0
Received ACK1
Received ACK2
Received ACK3
window is move to index 4
Sending packet 4: M4:a
Sending packet 5: M5:te
Sending packet 6: M6:st
Sending packet 7: M7: m
Received ACK4
Received ACK5
Received ACK6
Received ACK7
window is move to index 8
Sending packet 8: M8:es
Sending packet 9: M9:sa
Sending packet 10: M10:g
Sending packet 11: M11:e
Received ACK8
Received ACK9
Received ACK10
Received ACK11
window is move to index 12
Finishing sending all messages + gets all acks back
```

צד שרת :

```

Sending ack message ACK9#
Server recived message - M10:g
Received expected packet 10
Sending ack message ACK10
Server recived message - M11:e
Received expected packet 11
Sending ack message ACK11
Client is finished , Closing socket
Waiting for a client to connect...

```

```

Server recived message - M0:Th
Received expected packet 0
Sending ack message ACK0#
Server recived message - M1:iis
Received expected packet 1
Sending ack message ACK1#
Server recived message - M2: i
Received expected packet 2
Sending ack message ACK2#
Server recived message - M3:s
Received expected packet 3
Sending ack message ACK3#
Server recived message - M4:a
Received expected packet 4
Sending ack message ACK4#
Server recived message - M5:te
Received expected packet 5
Sending ack message ACK5#
Server recived message - M6:st
Received expected packet 6
Sending ack message ACK6#
Server recived message - M7: m
Received expected packet 7
Sending ack message ACK7#
Server recived message - M8:es
Received expected packet 8
Sending ack message ACK8#
Server recived message - M9:sa
Received expected packet 9
Sending ack message ACK9#
Server recived message - M10:g
Received expected packet 10
Sending ack message ACK10

```

מה שבעצם אנחנו רואים בצד השרת זה את החבילה שהתקבלה ואת הACK שהשרת שולח עלייה

ננתח את המידע באWireshark

No.	Time	Source	Destination	Protocol	Length	Info
817	3.76290	127.0.0.1	127.0.0.1	TCP	74	55100 → 12348 [SYN] Seq=0 Win=65536 Len=0 MSS=65536 SACK_PERM=1 TSval=2095431263 TSecr=0 WS=128
1017	3.76338	127.0.0.1	127.0.0.1	TCP	74	12348 → 55100 [SYN, ACK] Seq=0 Ack=1 Win=65536 Len=0 MSS=65536 SACK_PERM=1 TSval=2095431263 TSecr=2095431263 WS=128
1117	3.76331	127.0.0.1	127.0.0.1	TCP	66	55100 → 12348 [ACK] Seq=1 Ack=1 Win=65536 Len=0 TSval=2095431263 TSecr=2095431263
1219	98144.	127.0.0.1	127.0.0.1	TCP	66	55100 → 12348 [PSH, ACK] Seq=1 Ack=1 Win=65536 Len=30 TSval=2095433918 TSecr=2095431263
1319	98145.	127.0.0.1	127.0.0.1	TCP	66	12348 → 55100 [ACK] Seq=1 Ack=31 Win=65536 Len=0 TSval=2095433918 TSecr=2095433918
1469	43344.	127.0.0.1	127.0.0.1	TCP	67	12348 → 55100 [PSH, ACK] Seq=1 Ack=31 Win=65536 Len=1 TSval=2095403370 TSecr=2095433918
1569	43345.	127.0.0.1	127.0.0.1	TCP	66	55100 → 12348 [ACK] Seq=31 Ack=2 Win=65536 Len=0 TSval=2095483370 TSecr=2095483370
17110	8458.	127.0.0.1	127.0.0.1	TCP	71	55100 → 12348 [PSH, ACK] Seq=31 Ack=2 Win=65536 Len=5 TSval=2095524783 TSecr=2095483370
18110	8458.	127.0.0.1	127.0.0.1	TCP	66	12348 → 55100 [ACK] Seq=2 Ack=36 Win=65536 Len=0 TSval=2095524783 TSecr=2095524783
19110	8459.	127.0.0.1	127.0.0.1	TCP	71	55100 → 12348 [PSH, ACK] Seq=36 Ack=2 Win=65536 Len=5 TSval=2095524783 TSecr=2095524783
20110	8459.	127.0.0.1	127.0.0.1	TCP	66	12348 → 55100 [ACK] Seq=2 Ack=41 Win=65536 Len=0 TSval=2095524783 TSecr=2095524783
21110	8459.	127.0.0.1	127.0.0.1	TCP	71	55100 → 12348 [PSH, ACK] Seq=41 Ack=2 Win=65536 Len=5 TSval=2095524783 TSecr=2095524783
22110	8459.	127.0.0.1	127.0.0.1	TCP	66	12348 → 55100 [ACK] Seq=2 Ack=46 Win=65536 Len=0 TSval=2095524783 TSecr=2095524783
23110	8459.	127.0.0.1	127.0.0.1	TCP	71	55100 → 12348 [PSH, ACK] Seq=46 Ack=2 Win=65536 Len=5 TSval=2095524783 TSecr=2095524783
24110	8459.	127.0.0.1	127.0.0.1	TCP	66	12348 → 55100 [ACK] Seq=2 Ack=51 Win=65536 Len=0 TSval=2095524783 TSecr=2095524783
25110	8460.	127.0.0.1	127.0.0.1	TCP	71	12348 → 55100 [PSH, ACK] Seq=51 Ack=51 Win=65536 Len=5 TSval=2095524783 TSecr=2095524783
26110	8460.	127.0.0.1	127.0.0.1	TCP	66	55100 → 12348 [ACK] Seq=51 Ack=7 Win=65536 Len=0 TSval=2095524783 TSecr=2095524783
27110	8460.	127.0.0.1	127.0.0.1	TCP	71	12348 → 55100 [PSH, ACK] Seq=7 Ack=51 Win=65536 Len=5 TSval=2095524783 TSecr=2095524783
28110	8460.	127.0.0.1	127.0.0.1	TCP	66	55100 → 12348 [ACK] Seq=51 Ack=12 Win=65536 Len=0 TSval=2095524783 TSecr=2095524783
29110	8461.	127.0.0.1	127.0.0.1	TCP	71	12348 → 55100 [PSH, ACK] Seq=12 Ack=51 Win=65536 Len=5 TSval=2095524783 TSecr=2095524783
30110	8461.	127.0.0.1	127.0.0.1	TCP	66	55100 → 12348 [ACK] Seq=51 Ack=17 Win=65536 Len=0 TSval=2095524783 TSecr=2095524783
31110	8461.	127.0.0.1	127.0.0.1	TCP	71	12348 → 55100 [PSH, ACK] Seq=17 Ack=51 Win=65536 Len=5 TSval=2095524783 TSecr=2095524783
32110	8461.	127.0.0.1	127.0.0.1	TCP	66	55100 → 12348 [ACK] Seq=51 Ack=22 Win=65536 Len=0 TSval=2095524783 TSecr=2095524783
33110	8461.	127.0.0.1	127.0.0.1	TCP	71	55100 → 12348 [PSH, ACK] Seq=22 Ack=56 Win=65536 Len=5 TSval=2095524783 TSecr=2095524783
34110	8463.	127.0.0.1	127.0.0.1	TCP	71	12348 → 55100 [PSH, ACK] Seq=56 Ack=27 Win=65536 Len=15 TSval=2095524783 TSecr=2095524783
35110	8463.	127.0.0.1	127.0.0.1	TCP	66	55100 → 12348 [ACK] Seq=27 Ack=71 Win=65536 Len=5 TSval=2095524783 TSecr=2095524783
36110	8463.	127.0.0.1	127.0.0.1	TCP	71	12348 → 55100 [PSH, ACK] Seq=71 Ack=32 Win=65536 Len=0 TSval=2095524830 TSecr=2095524783
37110	8926.	127.0.0.1	127.0.0.1	TCP	76	12348 → 55100 [PSH, ACK] Seq=32 Ack=71 Win=65536 Len=10 TSval=2095524830 TSecr=2095524830
38110	8926.	127.0.0.1	127.0.0.1	TCP	66	55100 → 12348 [ACK] Seq=71 Ack=42 Win=65536 Len=0 TSval=2095524830 TSecr=2095524830
39110	8926.	127.0.0.1	127.0.0.1	TCP	71	55100 → 12348 [PSH, ACK] Seq=42 Ack=76 Win=65536 Len=5 TSval=2095524832 TSecr=2095524830
40110	8928.	127.0.0.1	127.0.0.1	TCP	66	55100 → 12348 [ACK] Seq=76 Ack=47 Win=65536 Len=15 TSval=2095524832 TSecr=2095524832
41110	8951.	127.0.0.1	127.0.0.1	TCP	71	12348 → 55100 [PSH, ACK] Seq=47 Ack=91 Win=65536 Len=5 TSval=2095524832 TSecr=2095524832
42110	8951.	127.0.0.1	127.0.0.1	TCP	66	55100 → 12348 [ACK] Seq=91 Ack=52 Win=65536 Len=0 TSval=2095524886 TSecr=2095524832
43110	8951.	127.0.0.1	127.0.0.1	TCP	76	12348 → 55100 [PSH, ACK] Seq=52 Ack=91 Win=65536 Len=10 TSval=2095524886 TSecr=2095524886
44110	9494.	127.0.0.1	127.0.0.1	TCP	66	55100 → 12348 [ACK] Seq=91 Ack=62 Win=65536 Len=0 TSval=2095524886 TSecr=2095524886
45110	9494.	127.0.0.1	127.0.0.1	TCP	66	55100 → 12348 [FIN, ACK] Seq=62 Ack=62 Win=65536 Len=0 TSval=2095524887 TSecr=2095524886
46110	9494.	127.0.0.1	127.0.0.1	TCP	66	12348 → 55100 [FIN, ACK] Seq=62 Ack=82 Win=65536 Len=0 TSval=2095524887 TSecr=2095524887
47110	9553.	127.0.0.1	127.0.0.1	TCP	66	55100 → 12348 [FIN, ACK] Seq=82 Ack=63 Win=65536 Len=0 TSval=2095524892 TSecr=2095524892
48110	9554.	127.0.0.1	127.0.0.1	TCP	66	12348 → 55100 [FIN, ACK] Seq=63 Ack=83 Win=65536 Len=0 TSval=2095524892 TSecr=2095524892
49110	9555.	127.0.0.1	127.0.0.1	TCP	66	55100 → 12348 [ACK] Seq=92 Ack=63 Win=65536 Len=0 TSval=2095524892 TSecr=2095524892

בקשת הגודל
המקסימלי של
ההודעה

העברת החבילות
ACK עליונים

סיום הקשר

נקרא להקלטת הזו q6 .

לא רואים את הבקשה שהלקוח מבקש מהמשתמש להכניס לו מידע משום שזה לא תקשורת שבין הלקוח לשרת אלא משהו שקורא באופן חיצוני.